UNIVERSIDAD NACIONAL DE LA PLATA

FACULTAD DE INFORMÁTICA

TESINA DE LICENCIATURA

TÍTULO: Observabilidad, una base para aplicaciones resilientes

AUTORES: Lucas Di Cunzolo

DIRECTOR: Miguel Angel Luengo

CODIRECTOR: Christian Adrian Rodriguez

ASESOR PROFESIONAL: Leandro Di Tommaso

CARRERA: Licenciatura en Informática

Resumen

El presente trabajo de grado investiga la observabilidad como base para el desarrollo de aplicaciones web resilientes en arquitecturas cloud-native. El estudio aborda la integración técnica de métricas, registros y trazas mediante el estándar OpenTelemetry y el stack desarrollado por Grafana. A través de la instrumentación de WordPress, Redmine y Wagtail, se analiza cómo la telemetría facilita el diagnóstico de fallas y la adaptabilidad del servicio. Finalmente, se demuestra el uso de señales de observabilidad para automatizar la resiliencia mediante el escalado horizontal.

Palabras Clave

observabilidad, resiliencia, Grafana, métricas, logs, trazas, telemetria, visualización, ambientes, twelve-factor app, Kubernetes, cloud-native, Wordpress, Redmine, Wagtail.

Conclusiones

La observabilidad no elimina los incidentes, pero proporciona las herramientas para comprenderlos, contenerlos y prevenirlos. A su vez, la observabilidad no es un producto que se instala, sino una disciplina que se construye a lo largo del ciclo de vida del software. Requiere decisiones en el diseño de las aplicaciones, en su instrumentación, en la arquitectura de recolección, en el almacenamiento de datos, y en la cultura del equipo que los interpreta y actúa en consecuencia.

Trabajos Realizados

Se desarrolló un conjunto de modificaciones y configuraciones que permitieron desplegar en un entorno cloud-native, como lo es un clúster de Kubernetes, tres aplicaciones desarrolladas en tres lenguajes diferentes. Mediante los datos recolectados se plantearon diferentes escenarios que permiten escalar horizontalmente, permitiendo así, disminuir la posibilidad de caídas, maximizando los recursos utilizados.

Trabajos Futuros

Como líneas de desarrollo a futuro se propone:

- *Uso de telemetria para la implementación de rollback automatizados.*
- *la incorporación de un service mesh como Istio o Linkerd permitiría obtener trazas y métricas de red a nivel de infraestructura.*

Fecha de la presentación: MARZO 2026

Observabilidad, una base para aplicaciones resilientes

por Lucas Di Cunzolo

A mis viejos, que me acompañaron en todo este camino,

A Maga, que me inspira día a día,

a la educación pública, que me formó,

a la UNLP, que me hizo quien soy…

Índice general

Objetivo **6**
Introducción **7**
Conceptos básicos acerca de Observabilidad **9**
Métricas 11
Ejemplos de métricas 12
Prometheus/OpenMetrics 12
Stack TICK/InfluxDB 13
Métricas estándar en sistemas informáticos 13
Registros o Logs 15
Ejemplos de logs 16
Common Log Format (CLF) 16
Logfmt 17
Syslog 18
RFC 3164 18
RFC 5424 19
Trazas 21
Ejemplos de trazas 21
Zipkin 22
Jaeger 23
Opentelemetry 25
Correlación de datos 30
Basado en el instante de tiempo 31
Basado en identificadores de requerimientos 32
Basado en exemplars 33
Ejemplo práctico basado en una alerta 34
Visualización y tratamiento de la información 34
Conocer la audiencia 35
Jerarquías visuales 36
Uso de métricas correctas 38
Correlación entre tableros 38
Factores sobre el almacenamiento de datos 39
Cardinalidad 39
Dimensionalidad 40
Grafana Mimir 41
Grafana Loki 42
Grafana Tempo 44
Retención 45
Grafana Mimir 45
Grafana Loki 46
Grafana Tempo 47

Muestreo (Sampling)..... 47
Grafana Loki..... 47
Grafana Tempo..... 48
Estado del arte..... 48
Conceptos aplicados actualmente..... 50
Arquitecturas conocidas..... 51
Problemáticas..... 54
Prácticas recomendadas..... 54
Desarrollo..... 54
12 factor apps..... 55
Integración contínua..... 57
Automatización de la compilación..... 57
Pruebas automatizadas..... 58
Revisión de código..... 58
Esquema de ramificación..... 59
Gestión de dependencias..... 62
Entornos aislados y replicables..... 62
Observabilidad..... 62
Métricas..... 62
Logs..... 63
Trazas..... 63
Monitoreo de caja negra..... 63
Despliegue y operaciones..... 64
Ambientes de despliegue..... 64
Estratégia de despliegue..... 65
Infraestructura como código..... 66
Monitoreo y registros del proceso y la aplicación..... 67
Rollbacks automatizados..... 67
Arquitecturas de observabilidad..... 67
Aplicación..... 69
Instrumentación..... 70
OpenTelemetry..... 70
Instrumentación manual..... 70
Redmine..... 70
Instrumentación automática..... 73
Wordpress..... 73
Wagtail..... 75
Fluentd..... 75
Contenerización..... 77
Despliegue en Kubernetes..... 78
Infraestructura del cluster..... 78
Chart de despliegue..... 79
Escalado automático con KEDA..... 79
Prueba de carga con Locust..... 80

Escalado observable en Grafana..... 81
Conclusiones.....82
Cumplimiento del objetivo..... 82
Sobre los pilares de la observabilidad.....82
Sobre OpenTelemetry como estándar unificador..... 83
Sobre el stack LGTM como alternativa open-source..... 83
Sobre la instrumentación de aplicaciones heterogéneas.....84
Sobre las prácticas recomendadas y la metodología Twelve-Factor..... 85
Sobre la observabilidad como base para la resiliencia..... 86
Bibliografía..... 88

Objetivo

El desarrollo de aplicaciones web ha experimentado un imparable crecimiento en diversos aspectos relacionados a la ingeniería de sistemas, la gestión de proyectos, la arquitectura de los sistemas, e incluso los propios paradigmas, lenguajes, frameworks y librerías no sólo evolucionan, sino que sientan precedente en tanto a las buenas prácticas.

Por su parte, la infraestructura donde estas aplicaciones o servicios web se despliegan, también han evolucionado, permitiendo la escalabilidad, alta disponibilidad y mejor desempeño de los sistemas. Este factor es de gran relevancia porque los sistemas toman dimensiones de gran magnitud donde cada componente puede escalar por separado y el mantenimiento de tales arquitecturas se complejizan.

El objeto de esta tesina es el de lograr identificar todo lo necesario para poder comprender y reconocer el desempeño de los sistemas en su ambiente productivo, de una forma agnóstica en tanto a la tecnología del desarrollo, a fin de poder anticipar potenciales problemas, definir bajo qué circunstancias escalar un servicio y minimizar los tiempos que toma descubrir qué parte de un gran sistema es la causante de problemas que perciben los usuarios finales.

De esta forma, se propone establecer una serie de marcos de trabajo para poder aplicar observabilidad sobre aplicaciones o servicios web, identificando las distintas responsabilidades a lo largo del ciclo de vida del software, para brindar mayor información relacionada con su desempeño y la calidad del servicio ofrecido.

Introducción

A lo largo de los últimos años podemos identificar cambios disruptivos en el campo de la ingeniería de software, tanto en lo que se refiere al desarrollo de aplicaciones, su despliegue en ambientes de ejecución, e incluso en la gestión de tales ambientes. El auge de las metodologías ágiles permitió a los equipos de desarrollo mejorar sus procesos hasta el punto de llegar a desplegar nuevas versiones de las aplicaciones, varias veces al día, sin que esto signifique un aumento en el esfuerzo por parte de los mismos.

Para poder afrontar este cambio en el paradigma, los equipos de operaciones tuvieron que adaptarse, y a raíz de ello surge a finales del 2009 el concepto de Devops[1]. A grandes rasgos, este concepto propone utilizar técnicas provenientes del desarrollo de aplicaciones para mejorar los procesos relacionados con la gestión de infraestructura, buscando la automatización de los mismos mediante la programación.

De forma similar, las arquitecturas de las aplicaciones fueron evolucionando a lo largo de los años, dando como resultado un abanico heterogéneo de oportunidades al momento de su diseño. Esta diversidad impacta en que coexistan varias arquitecturas diferentes, incluso en el mismo ecosistema de un organismo o empresa, por lo que afrontar soluciones de interoperabilidad puede ser una tarea complicada. Así han surgido diseños de arquitectura que han permitido simplificar estas cuestiones como es la arquitectura orientada a servicios (SOA), o los flamantes microservicios.

Estas tendencias tecnológicas, han evolucionado cada una de forma independiente, a veces convergiendo sus caminos en puntos que inevitablemente cruzan las líneas del desarrollo de software con la de gestión de las infraestructuras. Y la magnitud de estos sistemas crece con los usuarios que consumen estos servicios masivamente. Empresas como Netflix, Spotify, Shopify, Amazon, Whatsapp, Facebook, etc son precursoras en diseñar soluciones para ofrecer una calidad de servicio inmutable a pesar de la gran demanda de sus usuarios. Estos diseños, muchas veces propietarios, fueron luego liberados como software libre e implementados por herramientas que resuelven diferentes problemas de este inmenso desafío. Y es de esta forma que emergen nuevas tendencias, como fueron las bases de datos no SQL, la ingeniería del caos[2], service mesh[3], entre muchos otros más. Incluso la observabilidad fue un tema que emerge de estos movimientos masivos, donde empresas de tales magnitudes no pueden afrontar el desafío de ofrecer un servicio de mala calidad o reconocer cómo economizar en su infraestructura que demanda un alto costo en proveedores cloud.

La realidad es que Google con su excelente aporte a la comunidad de software libre en 2014, con su donación de Kubernetes[4] a la CNCF[5], abrió al mundo una nueva forma de gestionar la infraestructura, tanto on-premise como en cloud. Con más de 10 años de su liberación, hoy Kubernetes transformó las posibilidades de escalabilidad de aplicaciones, pudiendo simplificar aquellas tareas que eran un ideal en 2009 con el paradigma DevOps. En aquel entonces, solamente unos pocos lo hacían a través de plataformas como servicio (PaaS) como Heroku, simplificando el despliegue a los desarrolladores de aplicaciones.

Lo cierto es que hoy día es posible implementar GitOps[6], no sólo en plataformas como servicio cloud sino en servidores propios auto gestionados, cloud y on-prem, con herramientas que abstraen de muchas cuestiones y por ende, simplifica la implementación de plataformas ad-hoc en organizaciones de pequeña o mediana envergadura. Este hecho trajo consigo la necesidad de mantener entonces visibilidad sobre aplicaciones compuestas por múltiples servicios heterogéneos, cada uno representado por contenedores efímeros, donde a su vez, cada uno escala y donde cuestiones tan simples como visualizar los logs ya no es trivial.

Por último, debemos destacar que con la evolución de la tecnología, y el uso masivo de internet y celulares conectados 24x7, muchas aplicaciones requieren ofrecer un servicio con el 99,9% de disponibilidad. Este umbral pone la vara muy alta a quienes desarrollan y gestionan la infraestructura, ahora unidos por el mismo motivo: tener que dar pronta respuesta ante un problema. En general, los servicios se degradan cuando crece de forma abrupta la cantidad de usuarios que lo utilizan. Es justamente en estos momentos cuando la observabilidad ayuda a desarrolladores y operadores a identificar, y hasta prevenir, dónde ocurre un incidente.

En la prevención de incidentes es donde aparece un nuevo concepto, la resiliencia. Podemos decir que una aplicación es resiliente si puede, ante eventos nuevos, adaptarse de forma favorable para evitar la degradación del servicio. Para poder llegar a que una aplicación sea resiliente, se pueden implementar diferentes estrategias, como es el caso de arquitecturas distribuidas, donde se busca maximizar la disponibilidad de un servicio haciendo uso de diferentes locaciones. Idealmente, deberían estar geográficamente separadas para asegurar que, por ejemplo, un corte de luz masivo no afecte el servicio. Este tipo de estrategias son implementadas por los diferentes proveedores de nube, bajo el concepto de zonas y regiones.

Otra posible estrategia es implementar auto-sanado (del inglés, self-healing). Esto se hace mediante el uso de health checks que permiten conocer la salud del propio servicio, en conjunto con otros productos que permiten reiniciar el servicio ante un chequeo fallido.

Finalmente, la estrategia en la que nos vamos a enfocar en esta tesis, la adaptabilidad de servicio. Esto se refiere a, mediante diferentes indicadores, permitirle al servicio escalar, ya

sea vertical u horizontalmente. El escalamiento vertical consta de darle más recursos de hardware al sistema, ya sea CPU, memoria, disco, entre otras cosas. Esto lo que nos permite es sobrepasar posibles umbrales que nos encontremos ante el uso del servicio, de una forma simple. La principal contra que tiene este tipo de escalamiento, es que está limitada al hardware real que tengamos en el momento en el equipo en el que se encuentra ejecutándose nuestro sistema.

Por otro lado, tenemos el escalamiento horizontal, que consta de agregar nuevas instancias de nuestro servicio, ya sea en el mismo, o en otro equipo físico. Esto nos permite, a su vez, balancear la carga entre las diferentes instancias del servicio.

En este punto es importante remarcar que, los propios servicios de los que depende nuestro sistema pueden contar con sus propias estrategias de resiliencia. De nada nos sirve tener un servicio que aplique todas las estrategias de resiliencia, si ante un pico de uso, la base de datos no puede soportar un incremento en la cantidad de consultas. Por lo que, para que nuestro sistema realmente sea resiliente, la totalidad de la arquitectura debe serlo.

La tesina se organiza en capítulos. Inicialmente, se presentarán los conceptos fundamentales de la observabilidad. Luego, se abordará la visualización de datos, incluyendo la provisión de información desde diversas perspectivas y la presentación de estándares clave. Posteriormente, se analizará el estado del arte en arquitecturas de observabilidad para sistemas web e infraestructuras de despliegue. Se ofrecerán recomendaciones para el ciclo de vida del desarrollo y se detallarán los stacks de observabilidad empleados. Finalmente, se incluirán ejemplos concretos diseñados para la tesina, junto con casos reales que ilustran similitudes y diferencias entre negocios, para concluir con un resumen del análisis y futuras líneas de investigación.

Conceptos básicos acerca de Observabilidad

El desarrollo de aplicaciones y la ingeniería de software atacan una serie de problemáticas que tratan de maximizar la calidad del código, la cooperación, trabajo en equipo y demás cuestiones, que generalmente concluyen en una entrega de un release de una aplicación. Sin embargo, desde la perspectiva de los operadores, o quienes se dedican al delivery, despliegue o mantenimiento en producción de tales aplicaciones, nace un nuevo desafío: la necesidad de poner un nuevo release en producción. Hablar de despliegue ya dispara dos escenarios: podemos desplegar una aplicación por primera vez o actualizar una aplicación en un release a otro. Y en el segundo caso incluso puede ser necesario volver atrás o realizar un rollback.

Ahondamos en la actualización de un release. El mismo puede ser cambiar un artefacto por otro, pero a veces además requiere aplicar cambios en los modelos de datos. Puede pasar además, que la aplicación demande una disponibilidad que no admita la baja del servicio. Si este es el escenario, tenemos que pensar cómo y cuándo aplicar un parche al modelo de datos manteniendo el servicio online. Claro que acá se hace evidente la necesidad de que los desarrolladores consideren este requerimiento, antes de lanzar un nuevo release. Pero una vez hecho un cambio, es deseable poder saber si la aplicación mejora o empeora en términos del uso de recursos, en relación a la versión anterior. Sobre todo si el costo de mi infraestructura depende de ello.

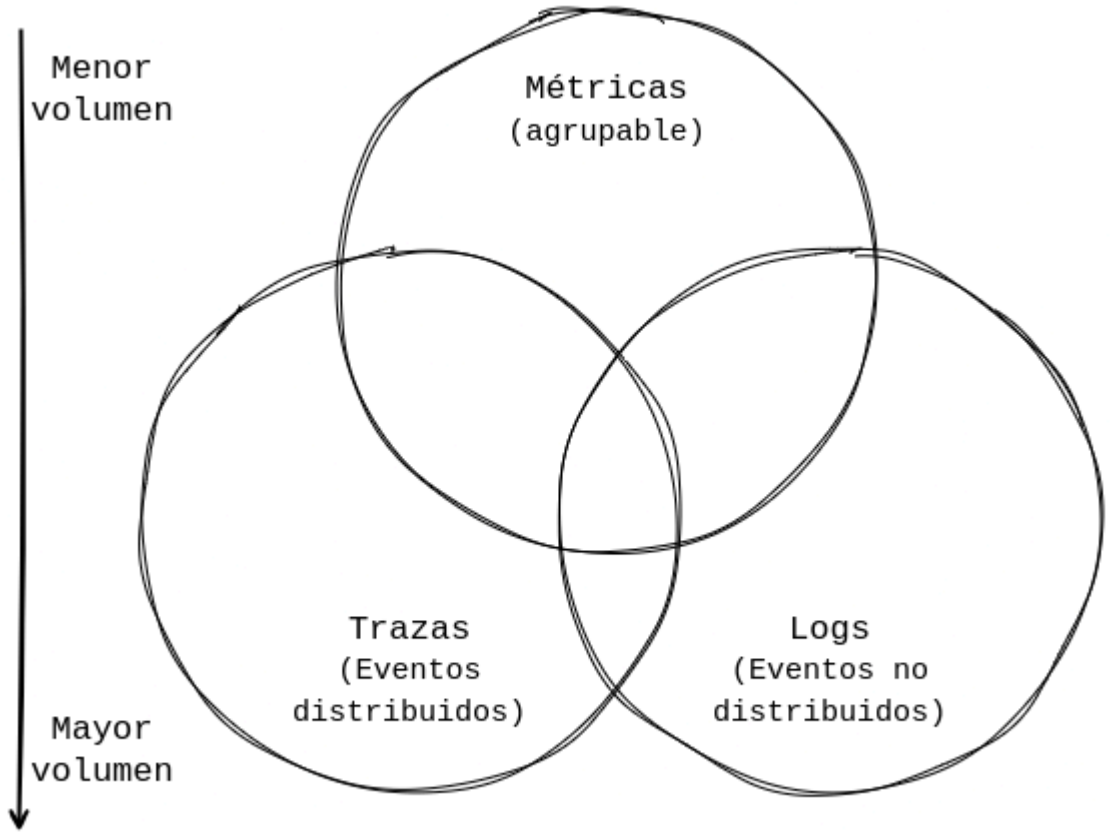
Como puede verse, hasta aquí simplemente hablamos del ciclo de vida de una aplicación, y solamente nos fuimos por una de las posibilidades de actualizarla. Y siendo optimistas, no consideramos la necesidad de un rollback. Así y todo, siempre aparece la necesidad de saber cómo funciona la aplicación mientras está siendo usada.

Una analogía que suele usarse entre quienes trabajan en observabilidad es: "¿saldrías a manejar por la ruta con el parabrisas lleno de barro y sin limpia parabrisas?". Claro que no es algo deseable. De igual forma, con las aplicaciones debería suceder lo mismo. Muchas veces los stakeholders son quienes primero preguntan por qué un servicio no funciona, y sin observabilidad es imposible dar una respuesta adecuada. Otra pregunta que no siempre tiene respuesta evidente es el costo real que tiene una aplicación para su despliegue, sea en dinero o recursos.

A modo de clarificar conceptos y desambiguar posibles malinterpretaciones, es importante definir qué significa observabilidad. Muchas veces se confunde su significado con la monitorización de servicios y su alertado, algo muy habitual entre los operadores, y que por años han resuelto dicha problemática con productos como Nagios[7] o Zabbix[8]. Pero como veremos a lo largo del documento, la observabilidad trata de un concepto mucho más abarcativo. Entonces podemos definirla como una propiedad deseable de los sistemas que le permite a las personas y a las máquinas observar, entender y actuar en respuesta al estado interno de un sistema. La forma que tienen los sistemas para exponer su estado es mediante señales. Las mismas, también conocidas como pilares, se pueden clasificar dentro de 3 categorías, las **métricas**, los **registros** *o logs,* y las **trazas**. Para almacenar y consultar estos datos, se usan unas bases de datos particulares, denominadas bases de datos de series de tiempo[9] (TSDB, de sus siglas en inglés, *Time Series Data Base*), las mismas son sistemas de almacenamiento diseñado específicamente para manejar datos organizados en función del tiempo. Estas bases de datos están optimizadas para insertar, consultar y analizar grandes volúmenes de datos temporales de manera eficiente.

En base a estos datos estructurados dentro de bases de datos, se busca obtener respuestas a diferentes preguntas que podemos llegar a tener en relación al funcionamiento

del sistema. Estas respuestas se obtienen relacionando y visualizando la información recolectada históricamente permitiendo conocer el estado interno de los sistemas, y tratando así de prever posibles incidentes.
Del párrafo anterior podemos notar la diferenciación de los conceptos datos e información. El primero hace referencia a las señales crudas sin ningún tipo de procesamiento, mientras que el segundo lo podemos definir como datos organizados de tal forma que permita tener un valor agregado.



Fuente: CNCF TAG Observability Whitepaper

Métricas

Las métricas son valores numéricos que representan el estado de un dato medido en un intervalo de tiempo determinado, que suele estar compuesta de 4 datos: un nombre identificatorio, el mismo en algunos casos suele ser un caso especial de atributo, un instante de tiempo o timestamp, un valor, y un conjunto de atributos, comúnmente denominados, *etiquetas* o *labels*.
Entre los datos que pueden ser representados por métricas, se pueden reconocer valores asociados al uso de recursos discretos de una máquina, como es el caso de la CPU, la memoria, el disco y la red, o por ejemplo peticiones HTTP atendidas de forma exitosa,

cantidad de errores, tiempo de respuesta, o incluso mediciones asociadas al negocio, datos climáticos, estado de sensores, entre muchos otros valores representables numéricamente.
Lo interesante de las métricas y las bases de datos TSDB, es que representan los datos de un recurso que se almacenan con una frecuencia determinada, por ejemplo el uso de la CPU cada 30 segundos. Esto permite que luego estos puntos puedan transformarse o visualizarse de forma de poder evidenciar qué sucede con un recurso.
Ahora bien, la estandarización de cómo representar una métrica es fundamental para simplificar su visualización e ingesta. La forma en que una métrica es enviada para luego ser almacenada, dependerá de la implementación, pero básicamente existen dos modelos: uno llamado push donde el propio servicio que genera una métrica envía los datos a la base de datos central, o pull donde la base de datos conoce los endpoints en que se exponen los servicios y con cierta frecuencia los recupera.

Ejemplos de métricas

Si bien cada producto tiene su forma de representar las métricas, podemos identificar un formato estándar, definido inicialmente por el producto Prometheus, y posteriormente adoptado por una serie de productos que se basaron en el en el motor y la API, como es el caso de Cortex[10], Thanos[11], Mimir[12], VictoriaMetrics[13], entre otros. Los mismos serán desarrollados en detalle en los siguientes capítulos. A su vez, este formato fue estandarizado en el proyecto OpenMetrics[14].

Prometheus/OpenMetrics

Es un formato que consta de un nombre de métrica, un conjunto de labels englobados entre llaves. Opcionalmente, pueden contener un timestamp al final de cada métrica, de no definirse, el timestamp tomado va a ser el del momento en que se recupera dicha métrica. Finalmente, pueden contener comentarios que explican el propósito de dicha métrica, y el tipo. Por ejemplo, a continuación se muestra los ciclos de CPU usados en una computadora usando el exporter oficial, llamado node-exporter

```
# HELP node_cpu_seconds_total Seconds the CPUs spent in each mode.
# TYPE node_cpu_seconds_total counter
node_cpu_seconds_total{cpu="0",mode="idle"} 267217.95
node_cpu_seconds_total{cpu="0",mode="iowait"} 113.08
node_cpu_seconds_total{cpu="0",mode="irq"} 0
```

```
node_cpu_seconds_total{cpu="0",mode="nice"} 1.52
node_cpu_seconds_total{cpu="0",mode="softirq"} 877.68
node_cpu_seconds_total{cpu="0",mode="steal"} 0
node_cpu_seconds_total{cpu="0",mode="system"} 7038.43
node_cpu_seconds_total{cpu="0",mode="user"} 2562.96
node_cpu_seconds_total{cpu="1",mode="idle"} 270009.45
node_cpu_seconds_total{cpu="1",mode="iowait"} 126.15
node_cpu_seconds_total{cpu="1",mode="irq"} 0
node_cpu_seconds_total{cpu="1",mode="nice"} 3.24
node_cpu_seconds_total{cpu="1",mode="softirq"} 56.12
node_cpu_seconds_total{cpu="1",mode="steal"} 0
node_cpu_seconds_total{cpu="1",mode="system"} 2175.1
node_cpu_seconds_total{cpu="1",mode="user"} 6670.94
```

Stack TICK/InfluxDB

InfluxDB utiliza un protocolo llamado "line protocol" el cual consiste en utilizar comas (",”) y espacios (" ") para separar los campos. A continuación se muestra la definición del protocolo, seguido de un ejemplo simple.

```
// Sintaxis
<measurement>[,<tag_key>=<tag_value>[,<tag_key>=<tag_value>]]
<field_key>=<field_value>[,<field_key>=<field_value>] [<timestamp>]

// Ejemplo
myMeasurement,tag1=value1,tag2=value2              fieldKey="fieldValue"
1556813561098000000
```

Métricas estándar en sistemas informáticos

Al abordar las métricas en sistemas informáticos, es posible identificar conjuntos ya establecidos que ofrecen una visibilidad valiosa, la cual será utilizada para los propósitos de esta tesis. Entre los conjuntos más destacados se encuentran:

- ***USE method***[15][16]: Es el nombre para referirse a 3 métricas que se suelen poder aplicar a cualquier recurso, aunque se suele ver aplicado para CPU, memoria, red y los discos en sistemas informáticos, las métricas en cuestión son:
    - *Uso*: es la cantidad de uso del recurso en cuestión. En el caso del CPU está dado en ciclos de CPU, en el caso de la memoria los bytes, en el caso de la red, los bytes de entrada y los de salida, en el caso del disco pueden ser los bytes y/o los ciclos de entrada/salida
    - *Saturación*: es el nivel de uso antes de que se produzca una degradación en el funcionamiento de ese recurso, en el caso del CPU está dado por la carga (load average), en la memoria los fallos de página, y en el disco la longitud de la cola de espera.
    - *Errores*: la cantidad de eventos de error presentes en el uso del recurso. No todos los recursos pueden representar esta métrica, y muchas veces se encuentra atado al soporte del hardware, en el caso de la CPU se puede interpretar como la cantidad de fallos en la caché del procesador, en el caso de la memoria la cantidad de errores al reservar la memoria virtual del proceso, en el caso de la red la cantidad de pérdida de paquetes, y en el disco la cantidad de errores de escritura.
- ***Golden signals***[17]: Son 4 métricas definidas por Google en su libro Site Reliability Engineering[18], en donde buscan definir cuales son la menor cantidad de métricas que permitan conocer el estado estado de nuestros sistemas. Las 4 métricas de las que hablan son bastante similares a las habladas en el método RED, con la diferencia que en este caso, se plantearon de forma genérica para varios tipos de sistemas. Las métricas en particular son
    - *Latencia*: el tiempo que le toma al servicio responder un requerimiento.
    - *Tráfico*: es un valor que muestra la cantidad de requerimientos servidos por el servicio. En este punto es importante diferenciar el tipo de servicio, para conocer de donde va a salir esta métrica. Por ejemplo en el caso de servicios HTTP, esta métrica está dada por la cantidad de requerimientos http por segundo, en servicios de streaming de audio está dada por la entrada/salida de la red, o las sesiones actuales, etc.
    - *Errores*: es un valor que muestra la cantidad de requerimientos con errores servidos por el servicio. En este caso, es importante remarcar que esta métrica va a estar dada tanto por errores explícitos, como puede ser el caso de códigos HTTP 500, sino de errores implícitos que nos permita conocer el estado de un servicio.

- *Saturación*: esta métrica mide el nivel de uso de los recursos limitados en un sistema. Esta métrica está dada completamente por el ecosistema y el funcionamiento de cada uno de los sistemas, y puede estar definida a partir de varios recursos, por ejemplo, un sistema que hace un uso intensivo de CPU y conexiones a un servicio en particular, puede definir su saturación en base a la cantidad de ciclos de CPU usados sobre el total asignado, y la cantidad de conexiones usadas sobre las posibles de un sistema. Muchos de los sistemas, suelen ver degradado su funcionamiento cuando se saturan, por lo que encontrar cuales son los recursos que limitan a nuestro sistema es un punto crucial al momento del análisis, y que puede ser determinante al momento de la toma de decisiones.

- ***RED method***[19]: Es el nombre para referirse a 3 métricas,
  - *Tasa*: del inglés **R**ate, la cantidad de requerimientos servidos por segundo.
  - *Errores*: del inglés **E**rrors, la cantidad de requerimientos con error servidos por segundo.
  - *Duración*: del inglés **D**uration. la cantidad de tiempo que toma cada requerimiento.

  Podemos pensar en este conjunto de métricas como la contraparte del USE method a nivel de servicio web. En el caso de RED, nos vamos a enfocar métricas que mejor representa el uso de un recurso web, como lo es la tasa de requerimientos, discriminado por cantidad total y cantidad de errores, y la duración que toman los requerimientos en ser respondidos.

Registros o Logs

Los registros, o del inglés, “Logs”, son un concepto familiar para quienes trabajan en TI para entender el comportamiento de los servicios. La definición formal de un registro se corresponde con la descripción de un conjunto de eventos discretos que suceden en un instante de tiempo en particular, y que pueden brindar información precisa relativa a las operaciones realizadas por las aplicaciones.

El formato de los registros a priori no parece ser algo estructurado. No al menos desde la perspectiva de quienes desarrollan y no han prestado especial atención a ellos. Sin embargo, todos los frameworks de desarrollo actuales permiten formatearlos con cierta estructura, aunque normalmente están representados por un instante de tiempo, y un valor referente al evento, donde dicho valor suele estar representado en texto plano, siendo en un formato estructurado o incluso de forma binaria. El formato en texto plano no

necesariamente cuenta con una estructura definida, por lo que pueden ser valores heterogéneos, que en ciertos casos deben estructurarse para su posterior uso. En el caso de los formatos estructurados, debemos reconocer si es un formato conocido, como por ejemplo json[20] o logfmt[21], para que los sistemas de ingesta y análisis de logs los interpreten de forma adecuada. De ser un formato estructurado no conocido, pero existe una definición del mismo, entonces mediante analizadores y filtros de conversión será posible recuperar toda la información. Existen varias técnicas para hacerlo, pero las que se destacan son el uso de expresiones regulares[22] o diferentes implementaciones de grok[23].

El formato de los logs tiene una estrecha relación con el servicio de syslog[24], ya que históricamente, syslog fue la forma que tenían los sistemas operativos para centralizar los logs de los diferentes servicios que se ejecutaban.

La necesidad de centralizar los logs no fue algo tan preocupante mientras los sistemas no escalaban. Pero hoy día, con las arquitecturas de microservicios o con la adopción de contenedores efímeros, la necesidad de llevar los logs a un lugar central donde puedan correlacionarse entre todas las réplicas es fundamental.

El principal problema que aparece cuando se centralizan los logs es la gran diferencia de formatos. Los más estabilizados son los accesos HTTP de servidores web, los de servidores de mail y otros servicios de ésta índole. Sin embargo, las aplicaciones o servicios web no suelen tener una estructura clara e incluso las excepciones en algunos lenguajes suelen representarse en múltiples líneas, siendo éste un problema habitual a resolver.

Cabe mencionar que uno de los principios que se establecen como buena práctica en la creación de contenedores, y que también se enuncia en uno de los 12 factores[25], es el de utilizar la salida estándar (STDOUT) y error estándar (STDERR) para escribir los registros. Es increíble la simplificación que ofrece un cambio tan simple como éste a la hora de instrumentar la centralización de logs.

Ejemplos de logs

A continuación podemos ver ejemplos de logs propios de servicios conocidos, en su formato estándar, es decir, que no cuentan con ninguna personalización al momento de configurar los propios servicios.

Common Log Format (CLF)

Este formato de log es el usado por dos de los servidores más usados en el mercado, como es el caso de NGINX y Apache. El formato está conformado de varios campos, separados por espacios, y, englobados por corchetes en el caso de la fecha, y por comillas en el caso

del requerimiento y el user agent. En los casos que el valor buscado no se pueda recuperar, se representa el valor usando el guión medio (“-”)

```
172.17.0.1 - - [24/Nov/2024:20:02:30 +0000] "GET / HTTP/1.1" 200 612 "-" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36" "-"
172.17.0.1 - - [24/Nov/2024:20:03:06 +0000] "GET / HTTP/1.1" 200 612 "-" "curl/7.88.1" "-"
```

El mismo se podría generalizar de la siguiente forma

```
<dirección remota> <identidad del usuario> <usuario remoto> [<fecha y hora requerimiento>] "<requerimiento>" <código de estado> <tamaño en bytes del cuerpo del requerimiento> "<valor del encabezado referer>" <user agent>

# O subdividiendo el requerimiento
<dirección remota> <usuario remoto> [<fecha y hora requerimiento>] "<método> <path> <version http>" <código de estado> <tamaño en bytes del cuerpo del requerimiento> "<valor del encabezado referer>" <user agent>
```

Volviendo al log de ejemplo, podemos identificar entonces que ambos requerimientos se hicieron al path “/”, usando el método “GET”, con la diferencia de que el primero se hizo usando un navegador mientras que en el segundo caso se hizo usando el comando “curl”, esto último lo podemos identificar en el user agent.

Logfmt

Es un formato de log estructurado, de fácil lectura y escritura diseñado para el router de Heroku, y estandarizado por primera vez por el paquete homónimo del lenguaje Go. Su fin es ser más simple de leer que un log en JSON, pero de igual facilidad al momento de analizar sintácticamente por sistemas, para su posterior inserción en bases de datos.
El formato consiste en un conjunto dinámico de campos, en el formato clave-valor, separados por espacios (“ ”), los valores pueden contener comillas para admitir casos con espacios.

```
baseUrl="/user/name" route=getName name=matthew msg="Get user name"
baseUrl="/user" route=getUser name=james msg="Get an user"
baseUrl="/resource" route=getResource name=peter msg="Get a resource"
```

En el ejemplo se pueden identificar tres líneas de log, con los atributos “baseUrl”, “route”, “name” y “msg”.

Syslog

Es un protocolo estándar usado para recolectar y reenviar logs y eventos en la red. Es ampliamente implementado en sistemas Unix y similares, aunque no se limita solamente a estas plataformas. El funcionamiento del protocolo está basado en un modelo cliente-servidor, en donde el cliente es el responsable de generar los logs y enviarlos al servidor, y el servidor es el responsable de recibirlos, ya sea mediante TCP o UDP, almacenar o reenviar los logs. El protocolo define una estructura la cual puede contener información tal como nivel de prioridad, instante de tiempo, el mensaje en sí, entre otros posibles campos. A su vez, el nivel de prioridad permite calcular cual es el recurso el cual generó el log, y cual es el nivel de severidad del mensaje. Esto se hace aplicando la siguiente fórmula.

```
Prioridad = Recurso * 8 + Severidad
```

Cada estándar define los posibles valores para recursos y severidad.

Actualmente el protocolo cuenta con 2 versiones estandarizadas en las RFC 3164 y RFC 5424, cada versión define un conjunto de campos que el cliente debe respetar para poder comunicarse con el servidor.

RFC 3164

Fue el primer estándar definido para el protocolo. El mismo consta de un formato en texto plano con una estructura fácilmente legible definida en la siguiente expresión

```
// Sintaxis
<PRIVAL>TIMESTAMP HOSTNAME TAG: MESSAGE
```

Por ejemplo:

```
// Ejemplo
<13>Oct 22 12:34:56 myhostname myapp[1234]: This is a sample syslog message
```

El ejemplo se interpreta de la siguiente forma

- Primero se puede identificar la prioridad, con el valor “13”, siguiendo la fórmula es posible calcular los sub-elementos de número del recurso, y la severidad del evento.

  ```
  Prioridad = Recurso * 8 + Severidad
  13 = 1 * 8 + 5
  ```
- Luego se identifica el instante en el tiempo, con el valor “Oct 22 12:34:56”
- En tercer lugar se identifica el nombre del hostname, con el valor “myhostname”
- En cuarto lugar se encuentra el nombre de la aplicación con el valor “myapp” y el identificador del proceso con el valor “1234”
- Finalmente, se identifica el mensaje del evento, con el valor “This is a sample syslog message. is the log message.”

RFC 5424

Es un protocolo que surge como una mejora al protocolo anterior,el RFC 3164. Entre las mejoras propuestas se encuentra agregar datos estructurados al evento, permitir la extensibilidad del protocolo, la estandarización del formato de la fecha para simplificar el parseo del campo.

El formato del protocolo está dado por la siguiente expresión

```
<PRIVAL>VERSION TIMESTAMP HOSTNAME APP-NAME PROCID MSGID [STRUCTURED-DATA] MESSAGE
```

Por ejemplo:

```
<13>1 2003-10-11T22:14:15.003Z myhostname myapp 1234 ID47 -
```

```
[exampleSDID@32473 iut="3" eventSource="Application" eventID="1011"] An
application event log entry...
```

El ejemplo se interpreta de la siguiente forma

- Primero se puede identificar la prioridad, con el valor “13”, siguiendo la fórmula es posible calcular los sub-elementos de número del recurso, y la severidad del evento.

  ```
  Prioridad = Recurso * 8 + Severidad
  13 = 1 * 8 + 5
  ```

- En segundo lugar, se identifica la versión, con el valor “1”
- Luego se identifica el instante en el tiempo, con el valor “2003-10-11T22:14:15.003Z”
- En tercer lugar se identifica el nombre del hostname, con el valor “myhostname”
- En cuarto lugar se encuentra el nombre de la aplicación con el valor “myapp”
- A diferencia del anterior protocolo, el identificador del proceso se encuentra separado del nombre de la aplicación, en este caso se puede identificar con el valor “1234”
- Luego podemos identificar el identificador del mensaje, con el valor “ID47”
- Entre corchetes se pueden identificar los datos estructurados. Los mismos cuentan con tipos definidos por el primer elemento entre corchetes, en formato “<nombre>@<número de empresa>”. En este ejemplo, el nombre sería “exampleSDID”, mientras que el número de empresa es “32473”. Luego lo siguen todos los valores estructurados en formato “clave-valor”, teniendo 3 elementos, “iut” con el valor “3”, “eventSource” con el valor “Application” y “eventID” con el valor “1011”.
- Finalmente, se identifica el mensaje del evento, con el valor “An application event log entry...”

Este protocolo fue extendido en la RFC 5425 para permitir el funcionamiento usando TLS para proveer una conexión segura en el transporte de los mensajes de syslog.

Finalmente, tanto el protocolo RFC3164, como el protocolo RFC5424 fueron extendidos para ser transmitidos sobre TCP/TLS en la RFC 6587.

Trazas

Las trazas son valores que describen eventos diferentes que conforman un flujo de un requerimiento HTTP desde que ingresa al sistema, hasta que es retornado al usuario. Sería una descomposición en pequeñas unidades de cada fragmento que procesa información y es relevante en un sistema. De esta forma, es posible observar cuáles son las demoras o en qué fragmento es dónde se produce un error. En un principio, puede asociarse con el profiling de aplicaciones, como el que ofrecen los frameworks de desarrollo durante el proceso de codificación, y en donde pueden observarse los tiempos involucrados de cada requerimiento explotado en tiempos que llevan las consultas a la base de datos, renderizado de vistas, así como también la cantidad de memoria involucrada y otros valores relevantes para esta etapa del desarrollo. En tanto a las trazas, el concepto es parecido, resumiendo la información considerada pero extendiendo la visibilidad a lo largo de complejas comunicaciones entre múltiples microservicios, teniendo una vistazo global de todos los pasos que dispara un requerimiento desde que ingresa por un servicio y cómo se va propagando por otros. Para lograr implementar esta característica de trazas correspondientes a uno o más servicios, es importante centralizar todos estos eventos para su posterior análisis.

Entre los datos que componen una traza podemos identificar a los Spans, que son la unidad mínima que representa cada una de las operaciones realizadas por el sistema y que consumen tiempo, como por ejemplo llamados a otras APIs o consultas a la base de datos, etc. Cada una de estas operaciones va a estar referenciada a su vez por la función o método que la produce, por lo que una vez armada la traza vamos a poder identificar potenciales cuellos de botella de forma simple. A su vez, si combinamos esto en un sistema que nos permita interrelacionar las trazas con métricas y registros, vamos a poder tener un panorama muy amplio y completo del funcionamiento interno de nuestros aplicativos, incluso cuando ellos se componen de múltiples microservicios.

Ejemplos de trazas

A continuación podemos ver ejemplos de trazas propias de servicios conocidos, en su formato estándar, es decir, que no cuentan con ninguna personalización al momento de configurar los propios servicios.

Las trazas se pueden representar en varios formatos los cuales van a estar condicionados por la herramienta seleccionada, pero podemos encontrar una forma simple de visualizarlas usando JSON.

Se verán ejemplos de los formatos más usados en el mercado, como es el caso de Zipkin[26], Jaeger[27] y OpenTelemetry[28]. Al momento de implementarlo, se utilizó Grafana

Tempo[29], por ser una cualidad del producto admitir los 3 formatos nombrados anteriormente.

Zipkin

A continuación se muestra una traza que consta de 2 spans, proveniente de la documentación oficial de Zipkin. El primer span es un requerimiento HTTP.
Por otro lado, el segundo span hace referencia a una consulta a la base de datos, teniendo en el campo de atributos, información contextual que nos permite entender desde qué parte del código está involucrada, hasta contar con la consulta en cuestión.


```
[
  {
    "traceID": "71699b6fe85982c7c8995ea3d9c95df8",
    "spans": [
      {
        "traceId": "5982fe77008310cc80f1da5e10147517",
        "name": "get",
        "id": "bd7a977555f6b982",
        "timestamp": 1458702548467000,
        "duration": 386000,
        "localEndpoint": {
          "serviceName": "zipkin-query",
          "ipv4": "192.168.1.2",
          "port": 9411
        },
        "annotations": [
          {
            "timestamp": 1458702548467000,
            "value": "sr"
          },
          {
            "timestamp": 1458702548853000,
            "value": "ss"
          }
    ]
  },
  {

    "traceId": "5982fe77008310cc80f1da5e10147517",
```

```
    "name": "query",
    "id": "be2d01e33cc78d97",
    "parentId": "ebf33e1a81dc6f71",
    "timestamp": 1458702548786000,
    "duration": 13000,
    "localEndpoint": {
      "serviceName": "zipkin-query",
      "ipv4": "192.168.1.2",
      "port": 9411
    },
    "remoteEndpoint": {
      "serviceName": "spanstore-jdbc",
      "ipv4": "127.0.0.1",
      "port": 3306
    },
    "annotations": [
      {
        "timestamp": 1458702548786000,
        "value": "cs"
      },
      {
        "timestamp": 1458702548799000,
        "value": "cr"
      }
    ],
    "tags": {
      "jdbc.query": "select distinct `zipkin_spans`.`trace_id` from `zipkin_spans` join `zipkin_annotations` on (`zipkin_spans`.`trace_id` = `zipkin_annotations`.`trace_id` and `zipkin_spans`.`id` = `zipkin_annotations`.`span_id`) where (`zipkin_annotations`.`endpoint_service_name` = ? and `zipkin_spans`.`start_ts` between ? and ?) order by `zipkin_spans`.`start_ts` desc limit ?",
      "sa": "true"
    }
  }
]
```


Jaeger

A continuación se muestra una traza de ejemplo en el formato de Jaeger. En este caso contamos solamente con un span de tipo HTTP.

```
{
  "data": [
    {
      "traceID": "71699b6fe85982c7c8995ea3d9c95df8",
      "spans": [
        {
          "traceID": "71699b6fe85982c7c8995ea3d9c95df8",
          "spanID": "3c191d03fa8be068",
          "operationName": "ftspanitron",
          "references": [],
          "startTime": 1680971616679312,
          "duration": 100000000,
          "tags": [
            {
              "key": "otel.library.name",
              "type": "string",
              "value": "manual-test"
            },
            {
              "key": "span.kind",
              "type": "string",
              "value": "server"
            },
            {
              "key": "otel.status_code",
              "type": "string",
              "value": "OK"
            },
            {
              "key": "internal.span.format",
              "type": "string",
              "value": "proto"
            }
          ],
          "logs": [],
          "processID": "p1",
          "warnings": null
        }
      ],
      "processes": {
        "p1": {
          "serviceName": "ft-test-with-curl",
          "tags": []
```

```
        }
      },
      "warnings": null
    }
  ],
  "total": 0,
  "limit": 0,
  "offset": 0,
  "errors": null
}
```


Opentelemetry

A continuación se muestran 2 trazas provenientes de un Wordpress instrumentado con OpenTelemetry. La primera traza es un requerimiento HTTP, y cuenta con información similar a la que podemos llegar a identificar en un log de un servidor web, como es el caso del path, método HTTP, user agent, entre otra información importante.
Por otro lado, la segunda traza hace referencia a una consulta a la base de datos, teniendo en el campo de atributos, información contextual que nos permite entender desde qué parte del código está involucrada, hasta contar con la consulta en cuestión.
Por fuera de los atributos, que son particulares de cada contexto, podemos identificar que comparten información tal como el inicio y el fin de dicha traza, un nombre identificatorio, y un conjunto de identificadores, que son usados tanto para interconectar trazas entre sí en forma de una cadena de trazas, en donde la traza que genera la creación de otra traza, pasaría a ser la traza padre, y comparten entonces ese identificador.


```
{
  "resourceSpans": [
    {
      "resource": {
        "attributes": [
          {
            "key": "service.name",
            "value": {
              "stringValue": "test-with-curl"
            }
          }
        ]
```

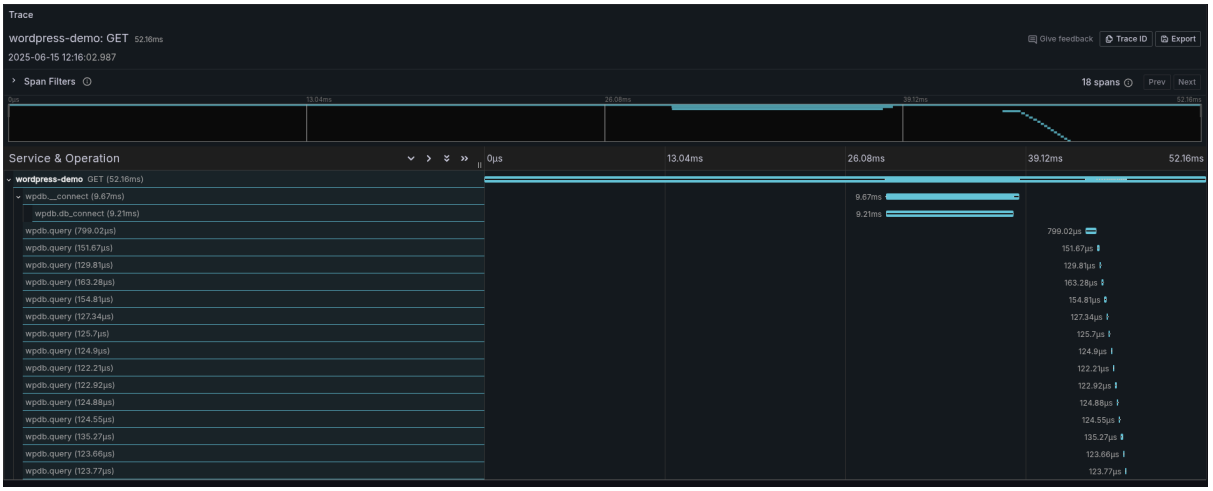
```
      },
      "scopeSpans": [
        {
          "scope": {
            "name": "manual-test"
          },
          "spans": [
            {
              "scope": {
                "name": "manual-test"
              },
              "spans": [
                {
                  "traceId": "a6f76593385af2cc5b1997cbd988d0b6",
                  "spanId": "eb753dc323f2dea2",
                  "parentSpanId": "",
                  "flags": 257,
                  "name": "GET",
                  "kind": 2,
                  "startTimeUnixNano": "1735573814541913498",
                  "endTimeUnixNano": "1735573815261083526",
                  "attributes": [
                    {
                      "key": "url.full",
                      "value": {
                        "stringValue": "http://wordpress.localhost
/wp-admin/ install.php"
                      }
                    },
                    {
                      "key": "url.scheme",
                      "value": {
                        "stringValue": "http"
                      }
                    },
                    {
                      "key": "url.path",
                      "value": {
                        "stringValue": "/wp-admin/install.php"
                      }
                    },
                    {
                      "key": "http.request.method",
```

```
                        "value": {
                          "stringValue": "GET"
                        }
                      },
                      {
                        "key": "network.protocol.version",
                        "value": {
                          "stringValue": "1.0"
                        }
                      },
                      {
                        "key": "user_agent.original",
                        "value": {
                          "stringValue": "Mozilla/5.0 (X11; Linux
x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0
Safari/537.36"
                        }
                      },
                      {
                        "key": "http.request.body.size",
                        "value": {
                          "stringValue": ""
                        }
                      },
                      {
                        "key": "client.address",
                        "value": {
                          "stringValue": "wordpress.localhost"
                        }
                      },
                      {
                        "key": "wp.is_admin",
                        "value": {
                          "boolValue": false
                        }
                      }
                    ],
                    "status": {}
                  },
                  {
                    "traceId": "a6f76593385af2cc5b1997cbd988d0b6",
                    "spanId": "70ac90ac44c7009e",
                    "parentSpanId": "eb753dc323f2dea2",
```

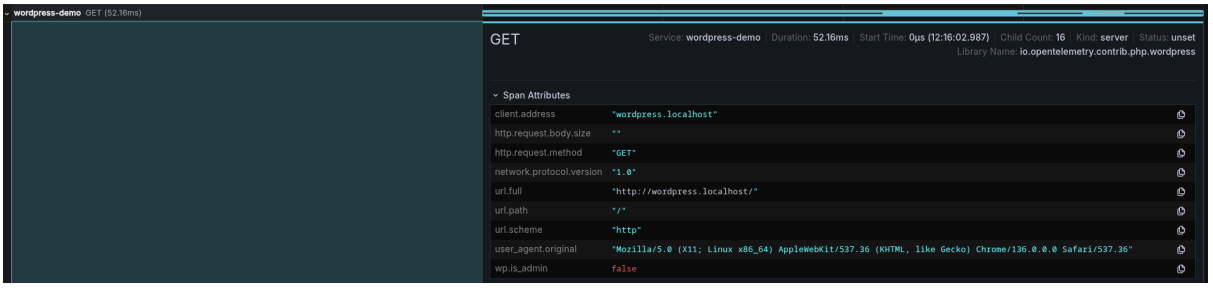
```
                    "flags": 257,
                    "name": "wpdb.query",
                    "kind": 3,
                    "startTimeUnixNano": "1735573815260767168",
                    "endTimeUnixNano": "1735573815260926789",
                    "attributes": [
                      {
                        "key": "code.function",
                        "value": {
                          "stringValue": "query"
                        }
                      },
                      {
                        "key": "code.namespace",
                        "value": {
                          "stringValue": "wpdb"
                        }
                      },
                      {
                        "key": "code.filepath",
                        "value": {
                          "stringValue":
"/var/www/html/wp-includes/class-wpdb.php"
                        }
                      },
                      {
                        "key": "code.lineno",
                        "value": {
                          "intValue": "2210"
                        }
                      },
                      {
                        "key": "db.statement",
                        "value": {
                          "stringValue": "SELECT option_value FROM
wp_options  WHERE option_name = 'date_format' LIMIT 1"
                        }
                      }
                    ],
                    "status": {}
                  }
                ]
              }
```

```
                ]
            }
        ]
    }
  ]
}
```

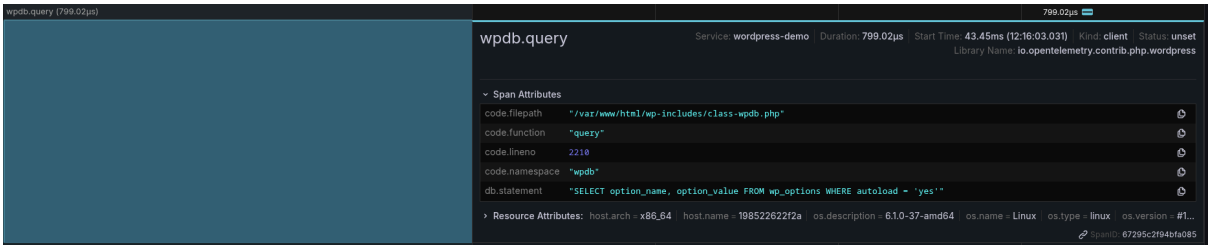
Esto se puede visualizar de forma más simple desde la interfaz de Grafana, en donde podemos ver las interdependencias entre diferentes spans. Cada span podemos decir que puede representar un paso necesario dentro de nuestra aplicación, por ejemplo, hacer una consulta a la base de datos. Este tipo de operaciones suele ser bloqueante, por lo que la aplicación no puede seguir su ejecución hasta tener la respuesta, generando una especie de ejecución en cascada.



Notar que, el primer span sería de tipo HTTP, ya que el primer paso al momento de responder operaciones del lado del cliente, es recibir un requerimiento HTTP. Si ampliamos los atributos de ese span, vemos algo similar a lo visto en el JSON anterior

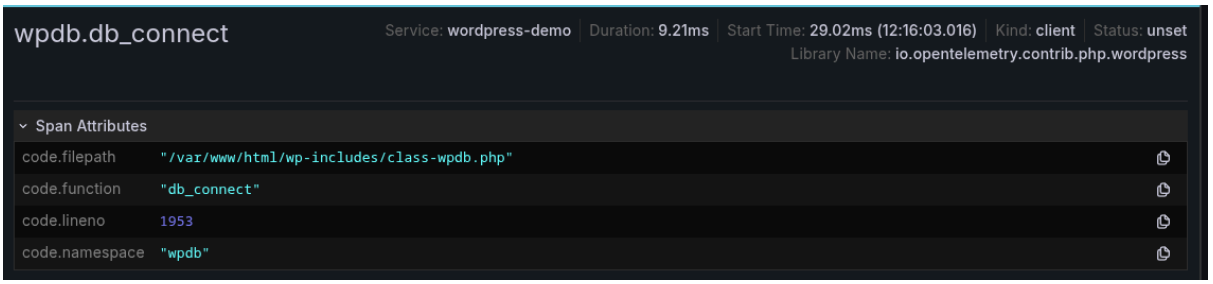


Por otro lado, si ampliamos cualquiera de las trazas llamadas “wpdb.query”, vamos a ver algo similar a lo visto en la segunda parte del JSON



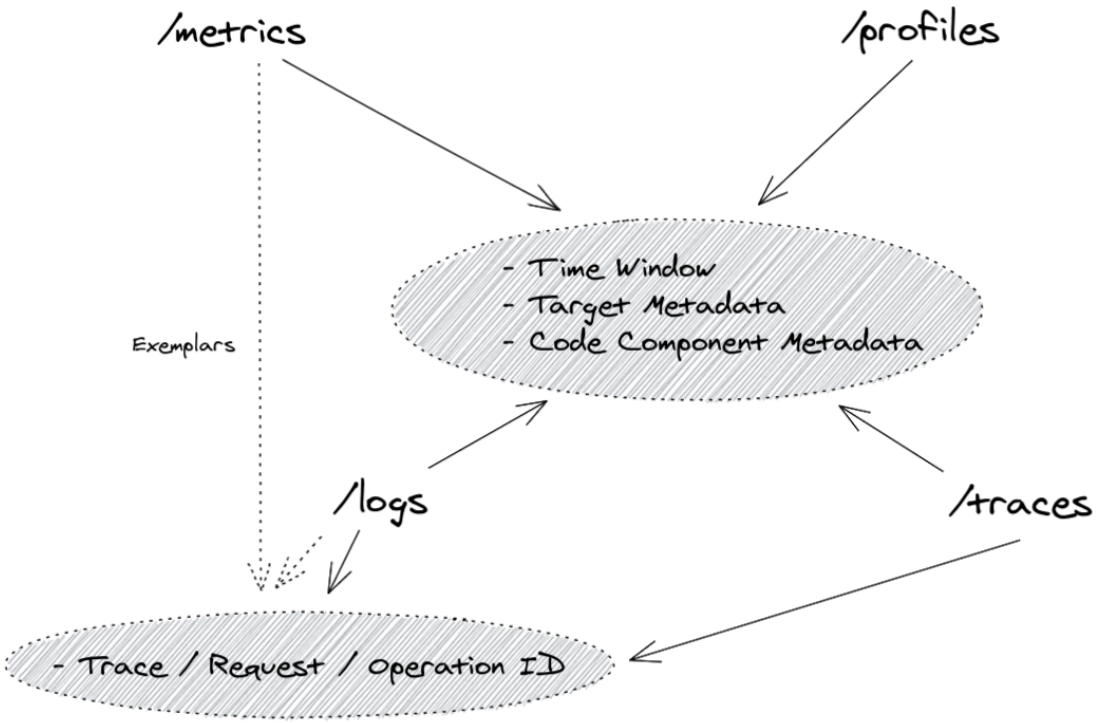
A su vez, de estos gráficos podemos sacar información importante para entender el funcionamiento de nuestra aplicación. Podemos ver que, dentro de las operaciones que nos exporta, la que más tardó fue la conexión a la base de datos, tomando 9.21ms, de los 52.16ms que tardó en generar una respuesta.

De igual forma que con los spans de tipo “wpdb.query”, podemos ingresar a sus atributos para recuperar más información contextual, como puede ser, el archivo PHP, el nombre de la función junto a la línea dentro del archivo donde se define dicha función



Correlación de datos

La correlación de datos permite relacionar de forma directa 2 o más pilares a través de diferentes mecanismos. De esta forma, es posible contar con una visión integral del estado interno de los sistemas desde diferentes perspectivas interrelacionadas entre sí, por lo que atender este punto es clave al momento de diseñar el sistema de observabilidad. Cabe remarcar que, para poder lograr la correlación de información, es necesario contar datos que todos los pilares almacenen y sirvan de nexo entre ellos. Caso contrario, la única forma de relacionarlos sería a través de ventanas temporales. En este sentido, si se logra entonces correlacionar los datos directamente, se simplifica la comprensión del estado global del sistema agilizando la navegación de la información, y en consecuencia ofrece una potente herramienta para el diagnóstico de incidentes. En general, los datos usados como nexo entre los pilares, son denominados metadatos, como por ejemplo puede ser el nombre del servicio, el identificador de una traza http, o en el caso de las métricas, los “Exemplars”.

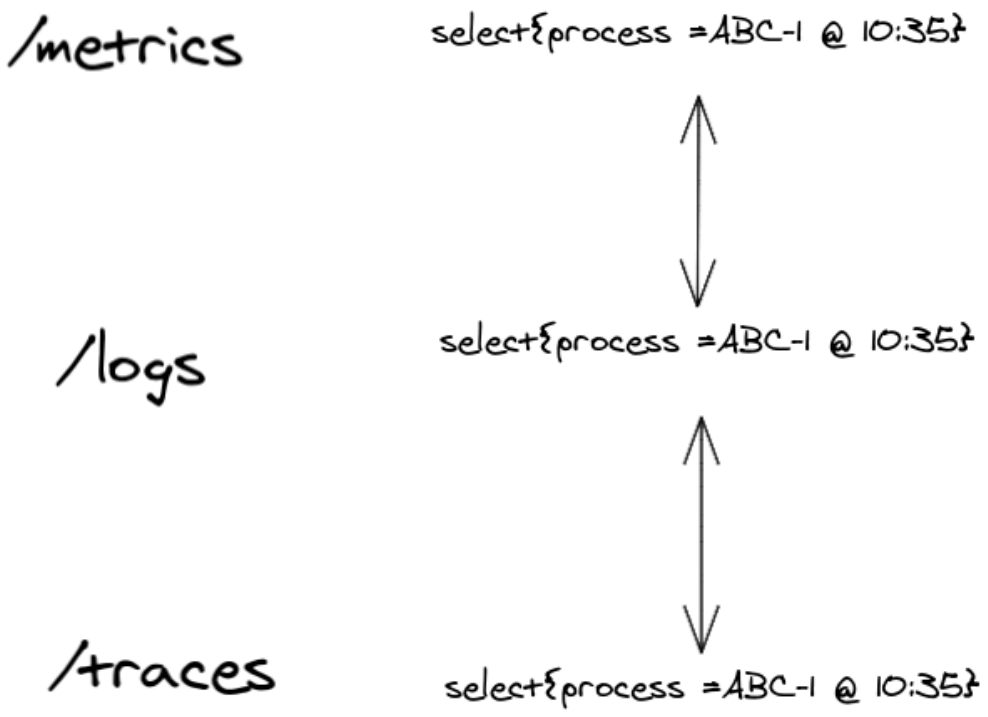


Fuente: CNCF TAG Observability Whitepaper

Como se puede notar en el gráfico, y mencionado anteriormente, las señales pueden correlacionarse por medio del instante de tiempo en que se generaron, o bien por la metadata extra que se le agrega.
A continuación se presentan diferentes flujos de navegación entre los pilares.

Basado en el instante de tiempo

Es el flujo más simple, el mismo no requiere instrumentación extra más que tener los instantes de tiempo en que se generaron las señales correctamente ingestadas. Podemos ver entonces que para cada señal es posible filtrar por el nombre del proceso en particular que estamos buscando, y acotar la búsqueda al instante de tiempo deseado.



Fuente: CNCF TAG Observability Whitepaper

Basado en identificadores de requerimientos

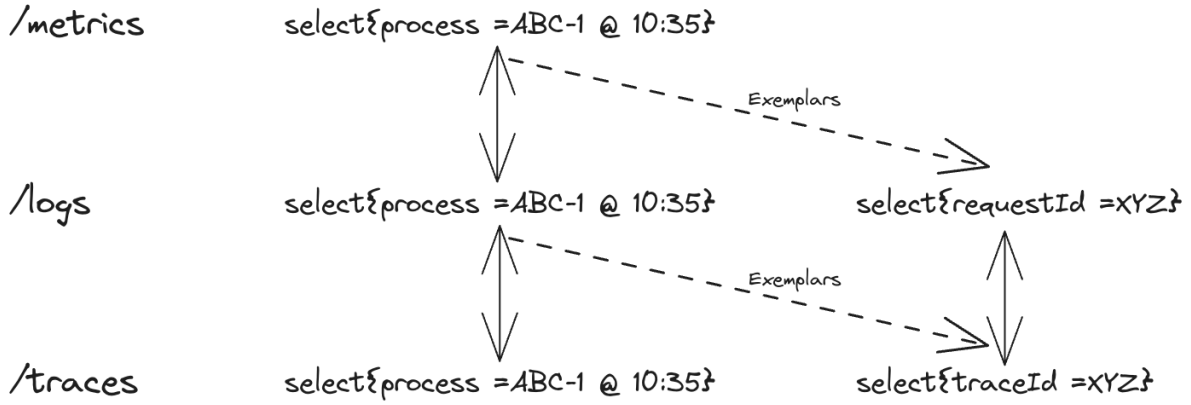
Este flujo sólo puede usarse para navegar entre trazas y logs, ya que en las métricas no tenemos forma de desagregar para identificar un requerimiento en particular. Para poder hacer uso de este flujo, es necesario que nuestro sistema agregue en los logs el identificador de la traza. De esta forma, vamos a poder filtrar solamente los logs que contengan el identificador de la traza, y de esta forma, relacionarlo con la traza particular que desee.

Puede observarse, que el conjunto de logs con trazas, siempre será un subconjunto de todos los logs.

Fuente: CNCF TAG Observability Whitepaper

Basado en exemplars

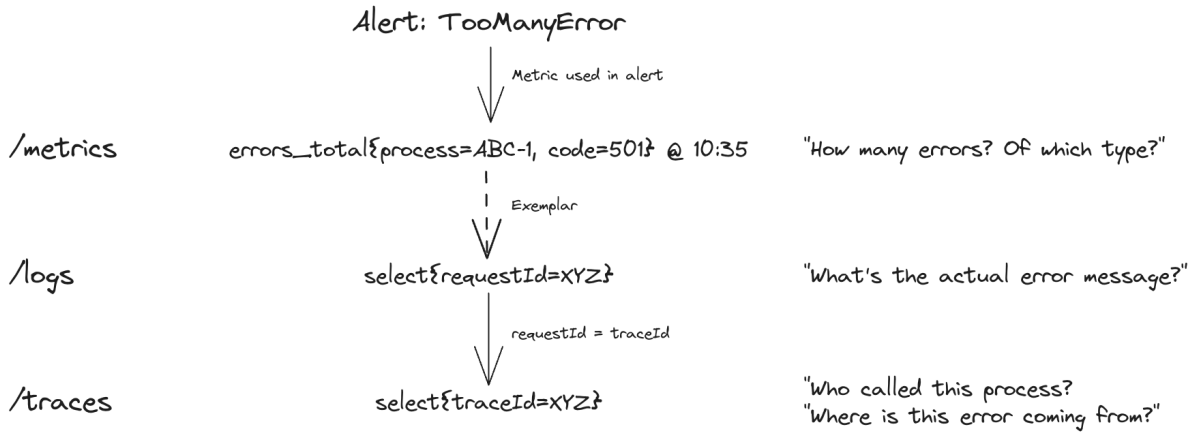
Es el flujo más completo, el mismo requiere contar con un dato extra proveniente de las métricas, llamado exemplar. Este dato suele estar ligado a una configuración extra al momento de instrumentar las señales en nuestro sistema, ya que requiere que tenga conocimiento de cuál es la traza a la que se debe referenciar. Esto nos permite identificar el valor de un requerimiento de forma aislada, por ejemplo, sabiendo la latencia particular de un requerimiento HTTP. De esta forma vamos a poder conectar en la visualización de la métrica, con una traza o un log a partir del identificador del requerimiento, permitiéndonos a su vez, extender el flujo anterior.



Fuente: CNCF TAG Observability Whitepaper

Ejemplo práctico basado en una alerta

Un ejemplo práctico de estos flujos puede ser basado en una alerta que se genera a partir de una métrica que marca la cantidad de errores 501 que se genera en el sistema. Al momento de recibir la alerta, vamos a ver cuantos errores y de qué tipo fueron. Esto nos puede llevar exactamente al log que genera el servicio para esos requerimientos. En el log vamos a poder identificar cual es el mensaje de error, y nos va a permitir ir directo a la traza en donde podemos desglosar paso a paso el requerimiento, y vamos a ver quien llamó a este requerimiento, y de donde viene el error. Alternativamente, de contar con los exemplars, podríamos ir directamente desde la métrica a la traza.



Visualización y tratamiento de la información

Hasta el momento se han presentado los diferentes datos que deben ser almacenados de forma centralizada. Ya sean métricas, logs o trazas, la información inerte en una base de datos o filesystem no sirven de mucho si no se procesa y ofrecen atajos visuales o alertas que las humanicen. Aquí es donde se pone en evidencia las necesidades que deben resolverse: la recuperación, transformación y almacenamiento de estos datos, para su consulta y la posterior visualización. Por lo pronto, hemos mencionado qué datos son relevantes y ejemplificado cada uno de ellos de forma aislada. Permitir visualizar estos datos de forma lineal es inadmisible por su volúmen.

Considerando por ejemplo un sitio de e-commerce que durante una fecha promocional alcanza valores por encima de los 7000 usuarios concurrentes, produciría en tal caso, una igual cantidad de registros o logs por segundo. Si pensamos que un ser humano procesa semejante volumen de logs en busca de problemas, es esperable que no realice grandes hallazgos.

Por tanto, las estrategias de visualización cumplen un rol fundamental a la hora de interpretar qué está sucediendo, sumarizando lo relevante y ayudándose de gráficos o

tableros. En el caso del e-commerce mencionado, sería más relevante que ver los logs en crudo, poder visualizar la cantidad total de registros por sobre aquellos que sean errores, y permitir luego filtrar únicamente aquellos registros con problemas. Con este ejemplo, se trata de graficar la necesidad de visualizar la información recolectada, siendo importante primero comprender su formato y la audiencia a quién se presentará esta información: no es lo mismo mostrar en una misma pantalla métricas de uso de recursos de un grupo de servidores junto con sus logs, que presentar las métricas de los requerimientos HTTP, junto a los logs y trazas de un conjunto de servicios, o incluso las métricas acerca del crecimiento y consumo de créditos por un grupo de servidores en un proveedor de cloud. Los usuarios interesados en cada uno de los ejemplos seguramente sean diferentes, y la visualización debe contemplar estas consideraciones durante el diseño.
Para visualizar los datos, se tomó como principal herramienta Grafana, ya que es la más ampliamente adoptada en el mercado por su versatilidad al momento de interactuar con diferentes fuentes de información, y el nivel de maduración tanto del proyecto, como de la comunidad que lo usa. A continuación definiremos los principales conceptos

- **Paneles**: Del inglés, panels, es una visualización de información en un formato particular. El mismo va a estar dado por el tipo de información que se quiera representar, cuál va a ser la audiencia, entre otras cosas.
- **Fuentes de información**: Del inglés, datasource, representa la conexión con diferentes bases de datos o servicios web.
- **Tablero**: Del inglés, dashboard, es un apartado que se crea dentro del sitio web de Grafana, y consta de uno o más paneles. A su vez, puede contar con variables que permitan desarrollar tableros genéricos que apliquen a varios contextos.

De igual forma que en el diseño de experiencia de usuario, cuando se diseñan tableros en grafana, se cuenta con diferentes puntos importantes que están estudiados y tienen buenos resultados. A continuación se detallan algunos puntos propuestos por la propia herramienta Grafana.

Conocer la audiencia

Una de las reglas más importantes al momento de diseñar tableros para visualizar información, es hacerlo con un propósito específico o un caso de uso en mente. El panel debe poder mostrar una progresión lógica de datos (como de lo general a lo específico o de lo grande a lo pequeño, responder a una pregunta o ambas cosas). Para lograrlo de manera

efectiva, es fundamental comprender a los usuarios o la audiencia a los que está dirigido dicho panel.

Por ejemplo, un operador tiene un objetivo muy diferente al usar un panel de Grafana en comparación con un gerente de producto. El primero probablemente necesite visualizaciones que le permitan identificar rápidamente si su servicio o sistema está funcionando según lo esperado y, en caso de degradación, ayudarle a localizar la causa del problema. Un gerente de producto, en cambio, podría usar un panel de Grafana para comprender cómo los clientes acceden e interactúan con el servicio. Esto nos deja con una gran variedad de casos de uso para los paneles de Grafana, tanto en entornos profesionales como personales.

Por tanto, antes de comenzar a diseñar un panel, es importante comprender el lugar del usuario al que va dirigido, definiendo sus objetivos y necesidades específicas en cuanto a la visualización de datos. Como parte de este proceso, también se debe considerar el método de entrega y el nivel de interacción requerido. Un panel presentado en un informe PDF, por ejemplo, no será interactivo, pero podría ser la mejor manera de enviar un informe mensual de desempeño al equipo directivo.

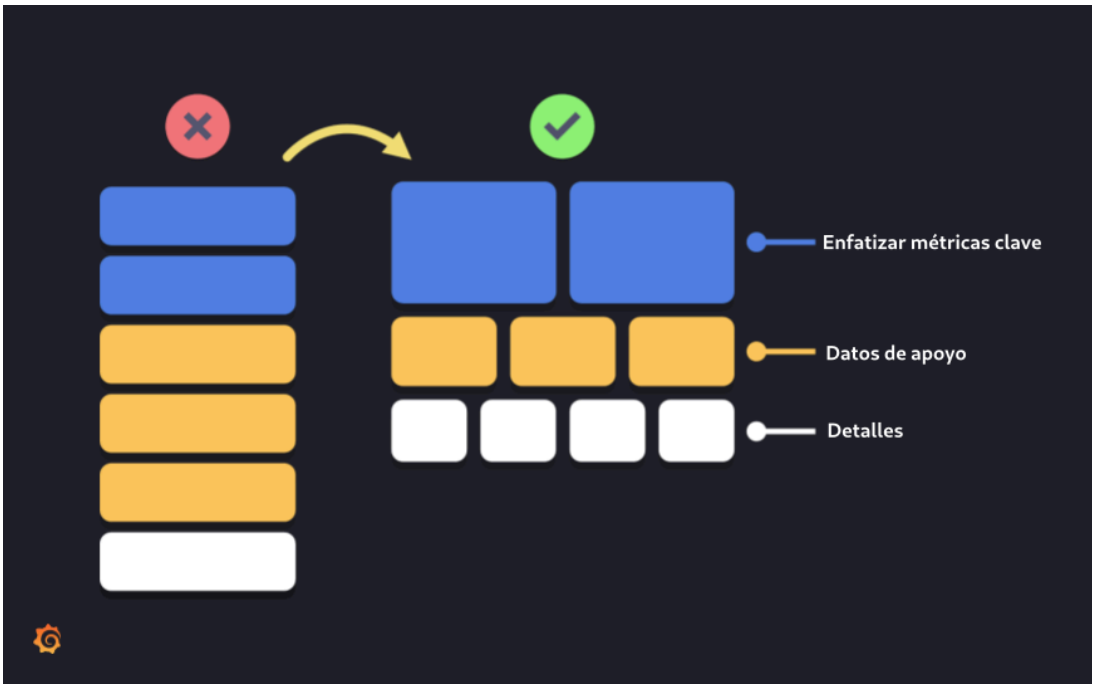
Jerarquías visuales

Una vez definida la audiencia y sus objetivos, es momento de pensar en cómo mostrar la información en el panel de manera más efectiva. En este punto es donde entra en juego el concepto de jerarquías visuales.
Una jerarquía visual es un patrón en el que algunos elementos destacan y atraen más atención que otros, lo que sugiere un orden de importancia. Al usar una jerarquía visual, se puede guiar al espectador hacia la información en la que se busca que se enfoque.

Existen varias categorías de jerarquía visual a considerar:

1. **Alineación**: Es el orden en el que se presenta o se organiza el contenido, lo que influye significativamente en la interacción. Los usuarios tienden a visualizar el contenido de una página —ya sea en un libro, en la web o en un panel de Grafana— siguiendo un patrón en "Z", de izquierda a derecha y de arriba hacia abajo. Conociendo esto, lo ideal sería colocar el contenido clave en la esquina superior izquierda del panel.

2. **Tamaño**: Cuanto mayor sea el tamaño de un componente, mayor será la percepción de su importancia. Por ejemplo, se pueden resaltar métricas clave en un panel de estadísticas grande y luego colocar datos de apoyo junto a otros detalles debajo en paneles más pequeños.



Fuente: blog grafana.com

3. **Color**: Naturalmente las personas se sienten atraídas por colores brillantes y saturados. Si el color de un objeto es visualmente distinto de su fondo, el espectador probablemente lo notará primero. Por lo tanto, se necesita usar paneles grandes en el centro del tablero (como un mapa geográfico) pero para enfatizar información en los paneles superiores, es posible usar el color para dirigir la atención del usuario hacia arriba.

   Finalmente, es importante recordar que muchos colores ya cuentan con un significado, como lo es el rojo con el error, el naranja o amarillo con la advertencia, y el verde con una operación exitosa.

4. **Forma**: Las formas complejas que sobresalen de su entorno también ayudan a captar la atención.

Una regla a tener en cuenta es que todo en el panel debe tener cierto nivel de importancia, sin que todo necesariamente se encuentre al mismo nivel. Acá es donde entran en juego, el poder de las jerarquías visuales, las mismas pueden ayudar a guiar a los usuarios hacia la información más relevante dentro del panel.

Por último, al pensar en estas señales visuales, la accesibilidad siempre es importante. Por ejemplo, para adaptarse a usuarios con daltonismo, evitar combinaciones de colores rojo-verde o rojo-marrón, ya que pueden ser difíciles de distinguir para ellos.

Uso de métricas correctas

Las métricas son otro aspecto importante a considerar en el diseño de paneles. Para elegir las métricas adecuadas a resaltar, se debe tener en cuenta al usuario objetivo y en la información que necesita para sacar conclusiones y tomar decisiones.

Es común que en los equipos de operaciones e ingeniería, se prioricen métricas que les ayuden a mantener SLOs[30](Service Level Objective) consistentes, mejorar la disponibilidad y reducir el MTTR (tiempo medio de reparación)[31]. Para estos tipos de usuarios de paneles, es útil utilizar el método RED, el mismo fue diseñado para observar la capa de servicio.
Otro conjunto de métricas interesantes para monitorear la infraestructura es el método USE, el cual fue diseñado para monitorear los recursos subyacentes, como CPU, memoria, disco y componentes de red, que soportan un servicio o aplicación

Correlación entre tableros

A diferencia de la correlación entre señales —que permite navegar de métricas a trazas o de trazas a logs dentro de un mismo evento—, la correlación entre tableros opera a un nivel superior: permite vincular distintos paneles de Grafana de forma que el usuario pueda transitar desde una vista de alto nivel hacia una vista de mayor detalle, o desde la perspectiva del servicio hacia la perspectiva de la infraestructura.

Grafana ofrece dos mecanismos principales para implementar esta navegación. El primero son los data links, que permiten adjuntar a cualquier punto de un panel un enlace que abre otro tablero pasando el valor del punto como parámetro. Por ejemplo, al hacer clic sobre el pico de latencia de un servicio en un gráfico de series de tiempo, es posible navegar automáticamente hacia el tablero de trazas de ese servicio filtrado en el instante de tiempo exacto donde ocurrió el evento.

El segundo mecanismo son los dashboard links, que asocian tableros entre sí mediante variables compartidas. Cuando un tablero declara una variable —como el nombre del

servicio o el entorno de despliegue—, esa variable puede propagarse al tablero de destino, garantizando que ambas vistas reflejan el mismo contexto. Esta técnica permite construir jerarquías de tableros: un tablero ejecutivo que muestra el estado global del sistema, un tablero operacional por servicio con métricas RED, y un tablero de infraestructura con métricas USE para los recursos subyacentes, todos navegables de forma coherente.

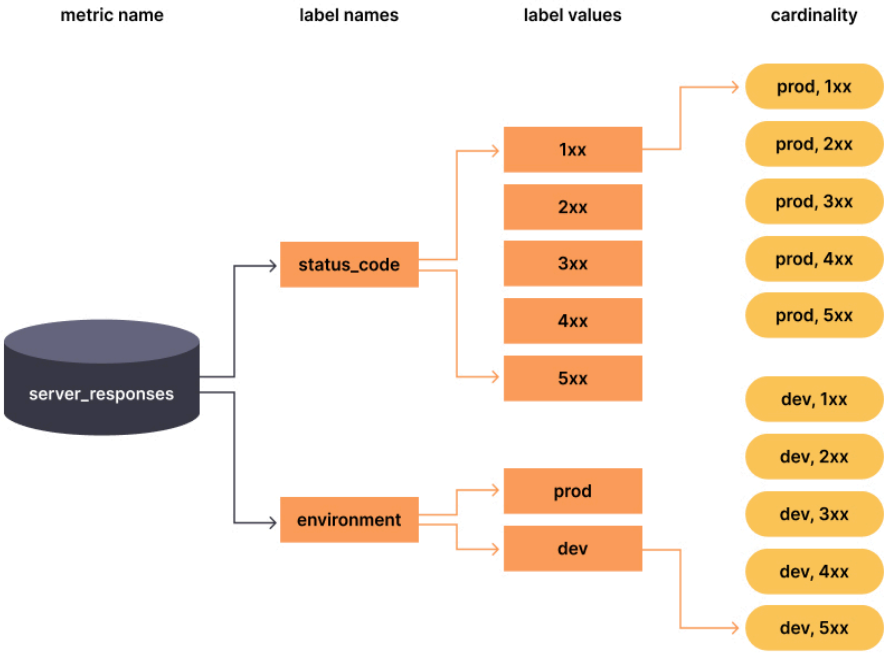
La correlación entre tableros es especialmente valiosa durante la investigación de incidentes, ya que reduce el tiempo necesario para transitar entre las distintas perspectivas del sistema —desde el síntoma visible hasta la causa raíz— sin perder el contexto temporal del evento bajo análisis.

Factores sobre el almacenamiento de datos

Los factores de importancia que consideramos pertinente definir en relación a las bases de datos son la cardinalidad, la dimensionalidad, retención y el muestreo (sampling). Dado que las bases de datos desarrolladas por Grafana fueron ampliamente adoptadas en el mercado, vamos a analizar los siguientes casos basándonos en Grafana Mimir[12] para métricas, Grafana Loki[32] para logs y Grafana Tempo para trazas.
La cardinalidad y la dimensionalidad están directamente relacionadas, y tienen un impacto en el rendimiento del sistema y en los costos de almacenamiento. A su vez, el tamaño en el almacenamiento de datos va a estar dado por la cardinalidad y la retención.

Cardinalidad

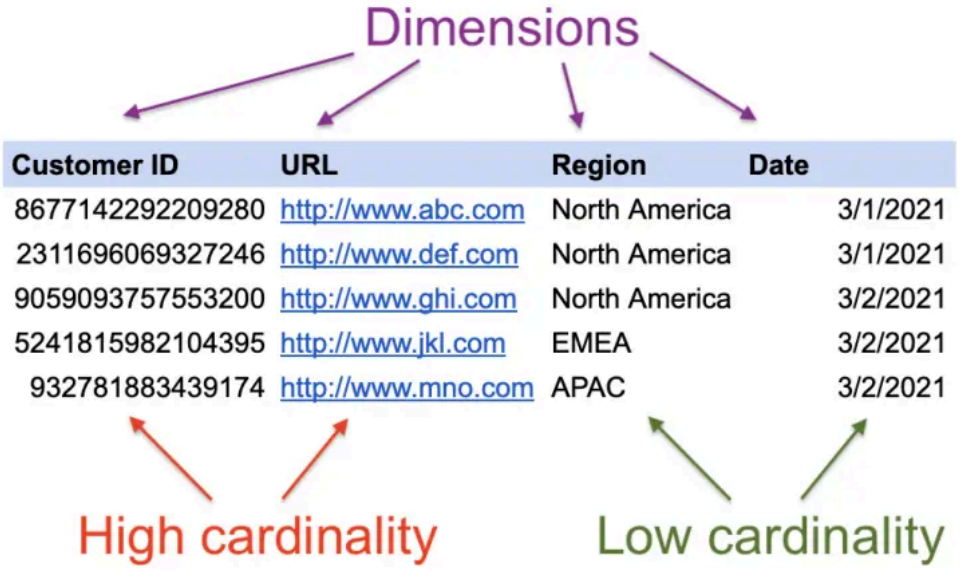
La cardinalidad representa la cantidad total de elementos únicos dentro de un conjunto de datos. La definición de unicidad de un elemento varía según el sistema de base de datos que se utilice. En el contexto de las métricas, la singularidad de un elemento se establece mediante la combinación particular de sus etiquetas. Cada combinación de etiquetas distinta se identifica como un elemento único dentro del conjunto de datos de la métrica. Por lo tanto, la cardinalidad es clave para evaluar la diversidad y el nivel de detalle presente en los datos que se están examinando. Una cardinalidad alta sugiere una gran variedad de elementos distintos, lo que puede indicar una mayor granularidad de los datos. Por el contrario, una cardinalidad baja implica que hay pocos elementos únicos, lo que podría señalar una menor diversidad o un nivel de agregación más alto en los datos. Comprender la cardinalidad de los datos es esencial para realizar análisis efectivos y para interpretar correctamente las métricas derivadas de esos datos.



Fuente: blog grafana.com

Dimensionalidad

La dimensionalidad hace referencia al número total de etiquetas diferentes de los elementos que se encuentran almacenados. Está directamente relacionado con la generación de índices de búsquedas, y con la cardinalidad final de los datos, por lo que también se encuentra relacionado con el espacio en disco necesario para almacenar la base de datos. En el caso de las 3 bases de datos de grafana, la dimensionalidad está dada por la cantidad de etiquetas (*labels*, en inglés). Cada etiqueta definida puede ser usada para el filtrado y la búsqueda de datos.



Fuente: blog newrelic.com

Grafana Mimir

Las métricas están basadas en el formato definido por OpenMetrics, descrito en el capítulo "Conceptos básicos acerca de Observabilidad".

```
# HELP node_cpu_seconds_total Seconds the CPUs spent in each mode.
# TYPE node_cpu_seconds_total counter
node_cpu_seconds_total{cpu="0",mode="idle"} 267217.95
node_cpu_seconds_total{cpu="0",mode="iowait"} 113.08
node_cpu_seconds_total{cpu="0",mode="irq"} 0
node_cpu_seconds_total{cpu="0",mode="nice"} 1.52
node_cpu_seconds_total{cpu="0",mode="softirq"} 877.68
node_cpu_seconds_total{cpu="0",mode="steal"} 0
node_cpu_seconds_total{cpu="0",mode="system"} 7038.43
node_cpu_seconds_total{cpu="0",mode="user"} 2562.96
node_cpu_seconds_total{cpu="1",mode="idle"} 270009.45
node_cpu_seconds_total{cpu="1",mode="iowait"} 126.15
node_cpu_seconds_total{cpu="1",mode="irq"} 0
node_cpu_seconds_total{cpu="1",mode="nice"} 3.24
node_cpu_seconds_total{cpu="1",mode="softirq"} 56.12
node_cpu_seconds_total{cpu="1",mode="steal"} 0
```

```
node_cpu_seconds_total{cpu="1",mode="system"} 2175.1
node_cpu_seconds_total{cpu="1",mode="user"} 6670.94
```

En este ejemplo, se puede identificar que la métrica “node_cpu_seconds_total” tiene una dimensionalidad de 2, pudiendo representar datos relativos a cada CPU de la máquina (en este ejemplo, los valores posibles son “0” y “1”), y teniendo también la dimensión de en qué modo se encontraba ese procesador específico, al momento de usar los ciclos de CPU (en este ejemplo, los valores posibles son “idle”, “iowait”, “irq”, “nice”, “softirq”, “steal”, “system” y “user”). Esto nos permitirá realizar consultas sobre esas 2 dimensiones, por ejemplo, podríamos recuperar la totalidad de ciclos de CPU usados por el CPU “0”, agrupado por modo

```
sum(node_cpu_seconds_total{cpu="0"}) by(mode)
# Ejemplo de respuesta
{mode="idle"} 11106252.41
{mode="iowait"} 8212.769999999999
{mode="irq"} 0
{mode="nice"} 17.53
{mode="softirq"} 21864.23
{mode="steal"} 409725.75
{mode="system"} 176404.32
{mode="user"} 525809.55
```

O bien podemos recuperar todos los ciclos de CPU consumidos en modo “idle” de todos los procesadores

```
sum(node_cpu_seconds_total{mode="idle"})
# Ejemplo de respuesta
{} 22213583.98
```

Grafana Loki

La forma de representar los logs, y el lenguaje de consulta están basados en el formato propuesto por Prometheus, posteriormente adoptado por OpenMetrics y definido como un estándar en representación de métricas. Está basado en una filosofía minimalista, en donde solo se busca generar índices de los datos estrictamente necesarios, reduciendo así la

cardinalidad de los datos, e impactando directamente en la reducción del espacio de almacenamiento necesario. A su vez, esto repercute en un mayor tiempo de ejecución de las consultas. Las etiquetas se configuran al momento de enviar los logs para ser insertados por la base de datos.

```
level=INFO path="/user/name/54" route=getName name=matthew msg="Get user name"
level=INFO path="/user/23" route=getUser name=james msg="Get an user"
level=INFO path="/resource/x" route=getResource name=peter msg="Get a resource by ID"
level=WARN path="/resource/x" route=getResource name=peter msg="Invalid ID, the value must be numeric"
level=INFO path="/user/update/23" route=updateUser name=james msg="Update user information"
level=ERROR path="/user/update/23" route=updateUser name=james msg="Failed to update user information"
```

Por ejemplo, para el log presentado anteriormente, podemos identificar diferentes campos que pueden ser de gran utilidad configurar como etiquetas, como puede ser el caso del nivel del log (“level”), la ruta del requerimiento (“path”), el método del controlador que responde al requerimiento (“route”), el nombre del usuario que ejecutó la acción (“name”), y el mensaje en texto libre (“msg”) que puede servir para conocer un poco más de información del estado de la aplicación. Todos estos campos, de definirse como etiquetas en el log, suman una nueva dimensión en la cual podemos consultar nuestros logs, y cada dimensión tiene una potencialidad diferente al momento de cómo va a impactar en la cardinalidad. Esto está dado por la cantidad de valores diferentes que pueden llegar a tener. Recordemos que cada etiqueta genera un índice invertido en donde vamos a poder consultar a nuestros logs, para simplificar el ejemplo, vamos a representar el índice invertido como un diccionario’

```
# Índice de level
{
  "INFO": [
      "level=INFO path=’/user/name/54’ route=getName name=matthew msg=’Get
```

```
user name'",
    "level=INFO path='/user/23' route=getUser name=james msg='Get an user'",
     "level=INFO path='/resource/x' route=getResource name=peter msg='Get a
resource by ID'",
         "level=INFO  path='/user/update/23'  route=updateUser  name=james
msg='Update user information'"
  ],
  "WARN": [
    "level=WARN path='/resource/x' route=getResource name=peter msg='Invalid
ID, the value must be numeric'"
  ],
  "ERROR": [
        "level=ERROR  path='/user/update/23'  route=updateUser  name=james
msg='Failed to update user information'",
  ],
}
```

Para la etiqueta “level” podemos notar que, aunque crezca la cantidad de logs, no se va a ver incrementado en gran medida la cantidad de entradas en el diccionario, se podrán agregar niveles como “TRACE”, o “DEBUG”, pero en la gran mayoría de los casos, la cantidad de valores únicos no va a superar la decena de valores. Por otro lado, si tomáramos como etiqueta a “path”, podemos proyectar que, dependiendo nuestra base de datos, la cantidad de valores únicos puede ascender a centenas, miles, o hasta millones, ya que dentro del path se pueden encontrar identificadores únicos de recursos. Esto puede ser un problema si no se comprende la morfología de los datos que se van a insertar en la base de datos.
A su vez, no todas las etiquetas deben estar definidas explícitamente en el log, al momento de enviar los datos a la base de datos se puede incluir información contextual, tal como el nombre de la aplicación, el host en el que se ejecuta, entre otros.

Grafana Tempo

De forma similar que en Grafana Mimir y Grafana Loki, en Grafana Tempo se usan etiquetas (tags) que proporcionan contexto y permiten filtrar y buscar trazas.

En Tempo, la "dimensionalidad" se relaciona con la cantidad de etiquetas únicas asociadas a las trazas. Cada traza puede tener varias etiquetas que describen aspectos del proceso,

servicio o componente que se está rastreando. Estas etiquetas son clave para buscar trazas específicas y entender el contexto de las operaciones.

Retención

Grafana Mimir

La retención de métricas en Grafana Mimir se refiere al período de tiempo durante el cual los datos de las métricas son almacenados y están disponibles para su consulta. Este concepto es fundamental para la gestión del ciclo de vida de los datos, el control de los costos de almacenamiento y la garantía de que la información histórica necesaria para el análisis y la resolución de problemas esté disponible.

Implicaciones y Aspectos Clave de la Retención:

- Disponibilidad de Datos Históricos: La retención define la ventana temporal dentro de la cual se pueden realizar consultas sobre las métricas. Un período de retención más largo permite analizar tendencias a largo plazo, realizar comparativas históricas y diagnosticar problemas que ocurrieron en el pasado.
- Gestión del Almacenamiento: La cantidad de datos de métricas generados puede ser significativa, especialmente en entornos de gran escala. Un período de retención más largo implica un mayor consumo de espacio de almacenamiento. Por lo tanto, la configuración de la retención debe equilibrar la necesidad de datos históricos con la capacidad y los costos del almacenamiento.
- Rendimiento de las Consultas: Si bien Mimir está diseñado para manejar grandes volúmenes de datos, un período de retención extremadamente largo podría potencialmente afectar el rendimiento de las consultas, especialmente aquellas que abarcan amplios rangos de tiempo. La forma en que Mimir organiza los datos en bloques y la configuración de los rangos de bloques influyen en este aspecto.
- Cumplimiento Normativo y Auditoría: En algunos casos, las regulaciones o las políticas internas pueden requerir la retención de datos durante períodos específicos con fines de auditoría o cumplimiento. La configuración de la retención debe alinearse con estos requisitos.
- Optimización de Costos: El almacenamiento de grandes cantidades de datos durante períodos prolongados puede generar costos significativos, especialmente si se utilizan soluciones de almacenamiento en la nube. Definir un período de retención adecuado ayuda a optimizar estos costos al evitar el almacenamiento innecesario de datos antiguos.

- Ciclo de Vida de los Datos: La retención es una parte integral del ciclo de vida de los datos de métricas. Implica decidir cuándo los datos dejan de ser relevantes o útiles y pueden ser descartados o archivados.

Grafana Loki

Grafana Loki no almacena los logs de la misma manera que otros sistemas de logs como Elasticsearch. En lugar de indexar el contenido de los logs, Loki indexa las etiquetas (labels). Esto significa que la "retención" en Loki se gestiona de manera diferente.

En el contexto de Grafana Loki, la retención se refiere al período durante el cual los logs están disponibles para ser consultados. Loki almacena los logs en trozos (chunks) en el almacenamiento de objetos (como S3, GCS o el sistema de archivos local). La retención en Loki se configura determinando cuánto tiempo se conservan estos trozos antes de ser eliminados.

La configuración de la retención en Loki implica:

- Tamaño de los chunks: Los logs se agrupan en trozos de un cierto tamaño o duración.
- Período de retención: Se define cuánto tiempo se mantienen estos chunks antes de ser eliminados.

La retención en Loki tiene implicaciones importantes:

- Costos de almacenamiento: Un período de retención más largo significa que se almacenan más datos, lo que puede aumentar los costos de almacenamiento.
- Disponibilidad de datos: Un período de retención más corto significa que los logs más antiguos se eliminan, lo que puede limitar la capacidad de realizar análisis históricos o depuración de problemas que ocurrieron hace mucho tiempo.
- Rendimiento de las consultas: La retención no afecta directamente el rendimiento de las consultas de la misma manera que en otros sistemas de logs, ya que Loki se centra en las etiquetas para indexar y no en el contenido completo de los logs. Sin embargo, tener una gran cantidad de datos almacenados aún puede influir en el rendimiento general del sistema.

Grafana Tempo

La retención de datos en Grafana Tempo se refiere al tiempo durante el cual los datos de las trazas son almacenados y están disponibles para su consulta. Es un aspecto clave de la gestión del ciclo de vida de los datos de las trazas, afectando tanto a la capacidad de realizar análisis históricos como a los costos de almacenamiento.

Al igual que con otras soluciones de almacenamiento de datos, definir una política de retención adecuada es crucial. Una retención más larga permite un análisis más profundo y la resolución de problemas que ocurrieron en el pasado, pero también conlleva mayores costos de almacenamiento. Por otro lado, una retención más corta puede reducir los costos, pero limita la capacidad de realizar análisis históricos.

Muestreo (Sampling)

Se refiere a la práctica de tomar una muestra representativa de datos en lugar de registrar cada evento o traza individual. Esto se hace para reducir la cantidad de datos que se almacenan y procesan, lo cual puede ser crucial para gestionar costos y mejorar el rendimiento, especialmente en entornos de alto volumen.

Grafana Loki

El servicio de Grafana Loki no cuenta con configuraciones para realizar sampling sobre los logs que ingesta, por lo que para aplicar este tipo de configuraciones vamos a tener que realizarlo sobre los productos encargados de recuperar los logs para luego enviarlos, como es el caso de Grafana Promtail. El mismo solamente admite el muestreo de logs basado en porcentajes. Es decir, podemos configurar para enviar solamente el 10% de los logs, descartando el otro 90%. Este tipo de enfoques tiene la contra de que no nos va a permitir tener el panorama completo de nuestro sistema, pero nos va a permitir ahorrar en almacenamiento y procesamiento de logs.
Por otro lado, se pueden configurar filtros basados en etiquetas para poder realizar este muestreo de forma más inteligente en base a los datos que se cuentan, pero requiere de un proceso más refinado de muestreo. El mismo también puede requerir mantenimiento a futuro, por lo que no es un enfoque transparente de aplicar.

Grafana Tempo

Tempo sí utiliza "sampling" para reducir la cantidad de trazas que se almacenan. Dado que las trazas pueden generar una gran cantidad de datos, el "sampling" es esencial para evitar sobrecargar el sistema y controlar los costos.

Tempo ofrece diferentes tipos de "sampling":

- Head-based sampling: La decisión de muestrear una traza se toma al inicio de la traza, generalmente en el primer servicio que recibe la solicitud. Si la traza se contempla, todos los spans subsiguientes de esa traza también se registran.
- Tail-based sampling: La decisión de muestrear una traza se toma después de que se completa la traza, lo que permite tomar decisiones basadas en la latencia o errores de la traza. Este tipo de "sampling" es más preciso, pero requiere más recursos ya que necesita almacenar temporalmente las trazas antes de decidir si se contemplan o no.
- Probabilistic sampling: Cada traza tiene una probabilidad predefinida de ser muestreada. Por ejemplo, si la probabilidad es del 10%, aproximadamente el 10% de las trazas se registrarán.

El "sampling" en Tempo se configura para equilibrar la necesidad de visibilidad con los costos de almacenamiento. Al ajustar la tasa de "sampling", se puede controlar la cantidad de datos de trazas que se almacenan.

Estado del arte

La observabilidad en ingeniería de software nace de la adaptación del concepto matemático homónimo creado por el ingeniero Rudolf Emil Kálmán en el año 1960[33], el cual lo definió como el nivel de precisión que se puede obtener al inferir el estado interno de un sistema solamente con información externa.
En estos últimos años se puede notar una creciente adopción en la aplicación del concepto de la observabilidad en los sistemas informáticos, en muchos casos acompañado por el crecimiento de comunidades que la impulsan, como es el caso de la comunidad que se formó en tornos productos como Prometheus y Grafana.
Uno de los casos más interesantes para analizar es el de la Cloud Native Computing Foundation (CNCF) como un pilar fundamental en el ecosistema de las tecnologías cloud

native. Nos interesa este caso particular, ya que es una fundación que nace promocionada por Google para alojar un producto que liberaron como Software Libre. El producto en cuestión era Kubernetes, un orquestador de contenedores en el que Google basó toda su infraestructura, y que le permitió escalar a nivel mundial. El segundo proyecto alojado por la fundación fue Prometheus en el año 2016, un sistema de monitoreo de código abierto que se convirtió en el sistema de facto para la recolección de métricas, para posteriormente desarrollar un estándar de transmisión de métricas en entornos cloud-native llamado OpenMetrics. Esto culmina en el inicio del desarrollo de múltiples backends de métricas a partir del núcleo de Prometheus, como es el caso de Thanos y Cortex, posteriormente devenido en Grafana Mimir.

En el caso de los Logs, podemos identificar varios stacks tecnológicos que cubren las necesidades para poder recuperar, transformar y almacenar los logs en un servicio para poder ser consumidos posteriormente. Dentro de los stacks más conocidos podemos encontrar a ELK, de sus siglas *Elasticsearch*, *Logstash* y *Kibana*. Siendo éste uno de los stacks con más trayectoria en el mercado. Cada uno de estos productos tiene un propósito específico, siendo Elasticsearch el backend en donde se almacenan todos los datos, Logstash el servicio donde se recuperan, procesan y envían los logs a nuestro backend, y Kibana el servicio de visualización. Actualmente podemos encontrar en muchos casos que se deja en desuso Logstash, reemplazandolo por diferentes productos, tales como *Filebeat* para la recolección y envío de Logs, mientras que el procesamiento de los mismos pasa a estar en una nueva funcionalidad de Elasticsearch llamado *Injest Pipelines*. Es interesante remarcar que todos los productos nombrados en este stack y sus variantes, son desarrollados por la organización de *Elastic*.

Por otro lado, y como una variante adoptada por productos de la CNCF tales como Openshift, podemos encontrar el stack EFK, de sus siglas *Elasticsearch*, *Fluentd/Fluentbit* y *Kibana*. Esta alternativa fue creada con la idea de cambiar uno de los elementos más ineficientes de su antecesor, Logstash, por productos que sean más Cloud-Native, y que permita escalar de forma más simple. Cabe destacar que esta alternativa puede ser combinada con la funcionalidad de *Injest Pipelines*, pasando a ser simplemente un *Shipper* de Logs. En este caso, *Fluentd* es un producto desarrollado en Ruby, que normalmente actúa como un punto central para el procesamiento y transferencia de logs, mientras que *Fluentbit* es un producto desarrollado en C, mucho más acotado, y que suele ser mucho más eficiente para recuperar los logs de diferentes fuentes.

Luego tenemos una alternativa más moderna, que es parte del conjunto de productos para Observabilidad de *GrafanaLabs* llamado *Grafana Loki*, usado normalmente en conjunto con *Promtail*, aunque puede usarse con *Fluentd/Fluentbit*, y hasta *Logstash/Filebeat*. Loki aparece como una alternativa mucho más liviana para almacenar y consultar logs que

Elasticsearch, ya que solamente se busca almacenar en índices invertidos, la estructura usada para la búsqueda de información en estas bases de datos, etiquetas que referencian al log, y no el dato completo como hace Elasticsearch por defecto. A su vez, este producto fue pensado para integrarse de forma directa con *Grafana* con el fin de visualizar los logs almacenados.

En este caso, no contamos con un estándar único para la transferencia de los logs, por lo que la arquitectura en muchos casos va a estar dada por los productos usados.

Por último, cuando hablamos de trazas, el pionero en traer el concepto fue Google, que con su paper "*Dapper, a Large-Scale Distributed Systems Tracing Infrastructure*"[34], estableció los conceptos que hoy conocemos: span, trace, context propagation, sampling. Luego, en 2012 Twitter implementó las ideas de Dapper en el primer producto opensource que llamó Zipkin. En 2015, Uber desarrolló su propia solución que llamó Jaeger, liberada recién en 2017. Éste último se creó considerando una arquitectura más escalable, pensada para tratar con volúmenes de datos muy grandes e incorporó sampling más sofisticado y una UI más completa. Eventualmente entró a la CNCF y se convirtió en el proyecto de referencia.

Con Zipkin, Jaeger, y varios vendors como Datadog y New Relic. Los desarrolladores debían elegir el vendor o producto antes de instrumentar el código, siendo esta cuestión una limitante importante. Así es como nació OpenTracing en el año 2019 como iniciativa de la CNCF, una API agnóstica del vendor para instrumentar código. La idea consistía en instrumentar contra la API de OpenTracing y después conectar con el backend que se deseara (Zipkin, Jaeger, etc.). Algo similar a los ORM usados en desarrollo.

Ya cuando todo parecía haberse estabilizado en el mundo de las trazas, en 2018 Google y Microsoft respondieron con OpenCensus, que además de tracing incluía métricas. Por tanto, competía directamente con OpenTracing, creando confusión en la comunidad.

La convergencia definitiva fue en 2019, cuando la CNCF fusionó OpenTracing y OpenCensus en OpenTelemetry (OTel), convirtiéndose en el estándar de facto. OTel unifica trazas, métricas pero además incorporó logs, y la posibilidad de seguir extendiendo las fuentes de información en una generalización que llamaron "señales". Actualmente se encuentra en desarrollo la incorporación del profiling como una nueva señal. Su funcionamiento se basa en OTLP protocol (OpenTelemetry Protocol) para exportar telemetría, y prácticamente todos los vendors y backends lo soportan hoy día.

Conceptos aplicados actualmente

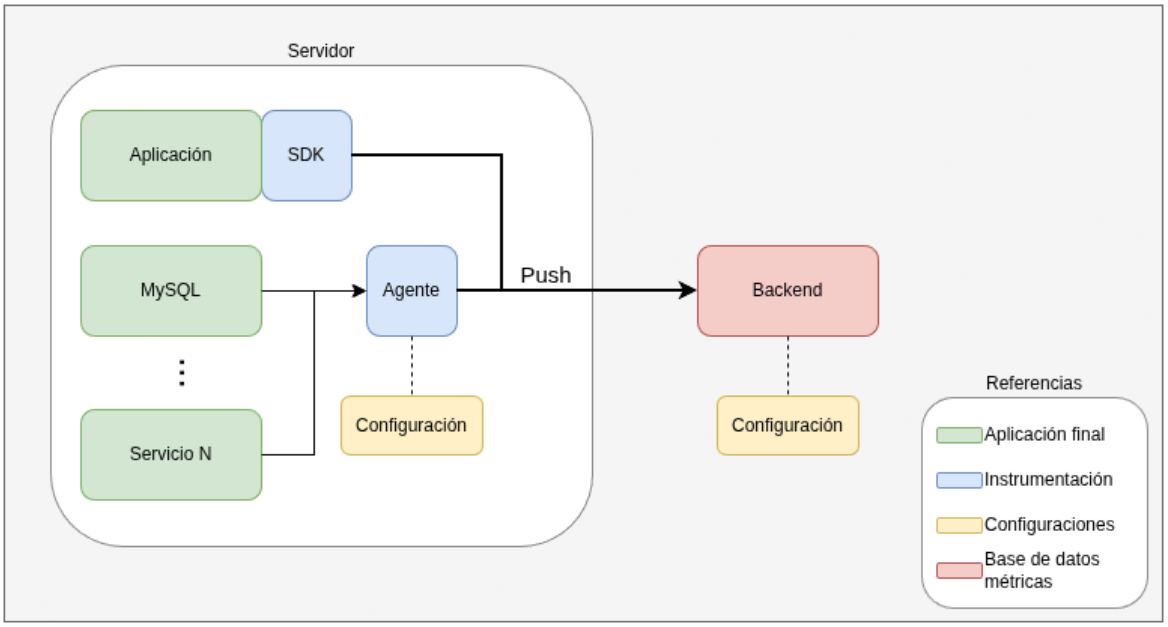
Podemos notar entonces que, gran parte de la industria empezó a transicionar hacia la estandarización, al menos, de los mecanismos de transferencia para generar un ecosistema

cooperativo que se nutre de la experiencia de varias empresas y organismos que deciden contribuir al software libre y a la CNCF. Casos como el de OpenMetrics, y OpenTelemetry nos permiten aprovechar al máximo los beneficios de la estandarización, permitiendo integrar gran parte de los productos del mercado a una única arquitectura.

Arquitecturas conocidas

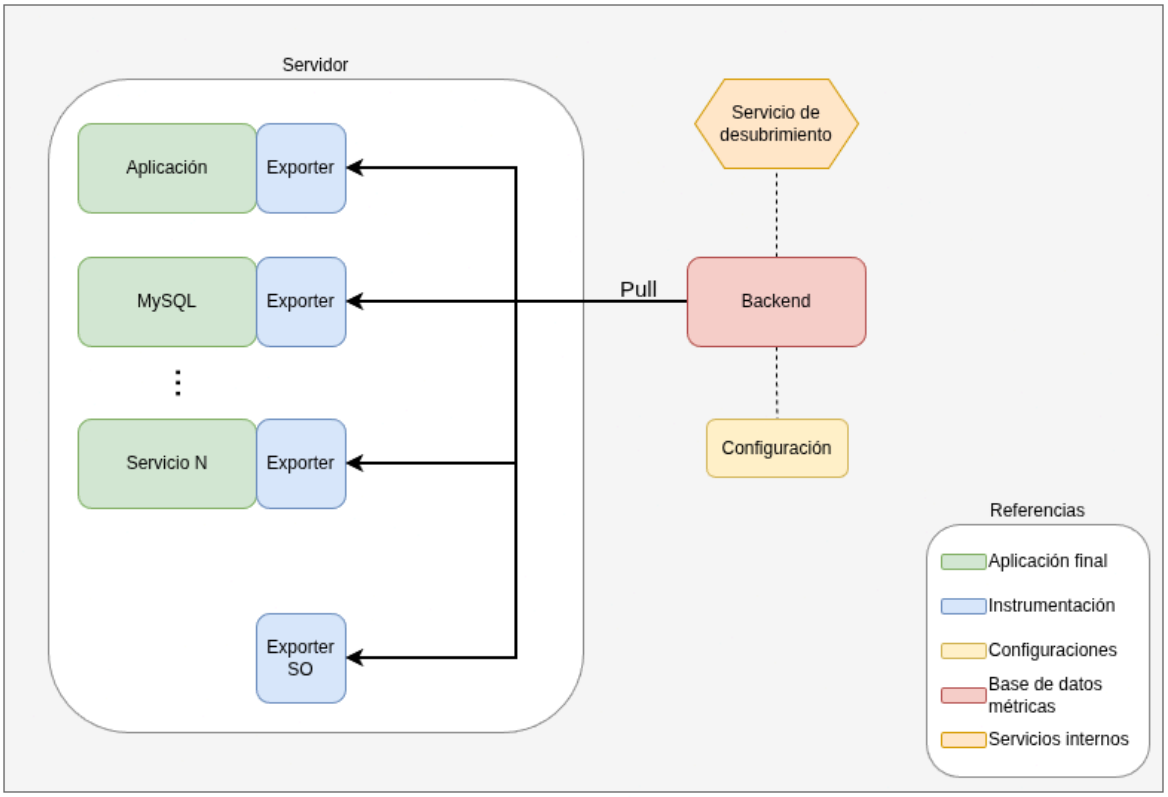
Al momento de clasificar las arquitecturas, podemos notar una primera categoría basada en la metodología usada para la recolección de los datos. En este caso, nos vamos a referir a arquitecturas de productos de métricas, ya que los productos que trabajan con registros y trazas suelen hacerlo en la modalidad Push, ya que suelen trabajar integrados con productos como logstash y fluentbit/fluentd.

Cuando definimos las arquitecturas, podemos identificar aquellas en las que un servidor se encarga de recibir los datos de diferentes agentes. A estas arquitecturas se las suele conocer como *Push*, normalmente en este tipo de arquitecturas la lógica y configuración se suele definir en cada uno de los agentes, por lo que podemos decir que son arquitecturas en la que crece la complejidad de mantenimiento a medida que crecen en tamaño. Uno de los principales exponentes de esta arquitectura es el stack tecnológico TICK (***T**elegraf, **I**nflux, **C**ronograf y **K**apacitor*), dedicado a la recolección, tratado y visualización de métricas basadas en el tiempo.



En contraste de las arquitecturas Push podemos identificar aquellas que cuentan con un servidor central, el cual concentra toda la configuración necesaria para recuperar los datos de cada uno de los agentes, a la que vamos a llamar arquitecturas *Pull*, normalmente este

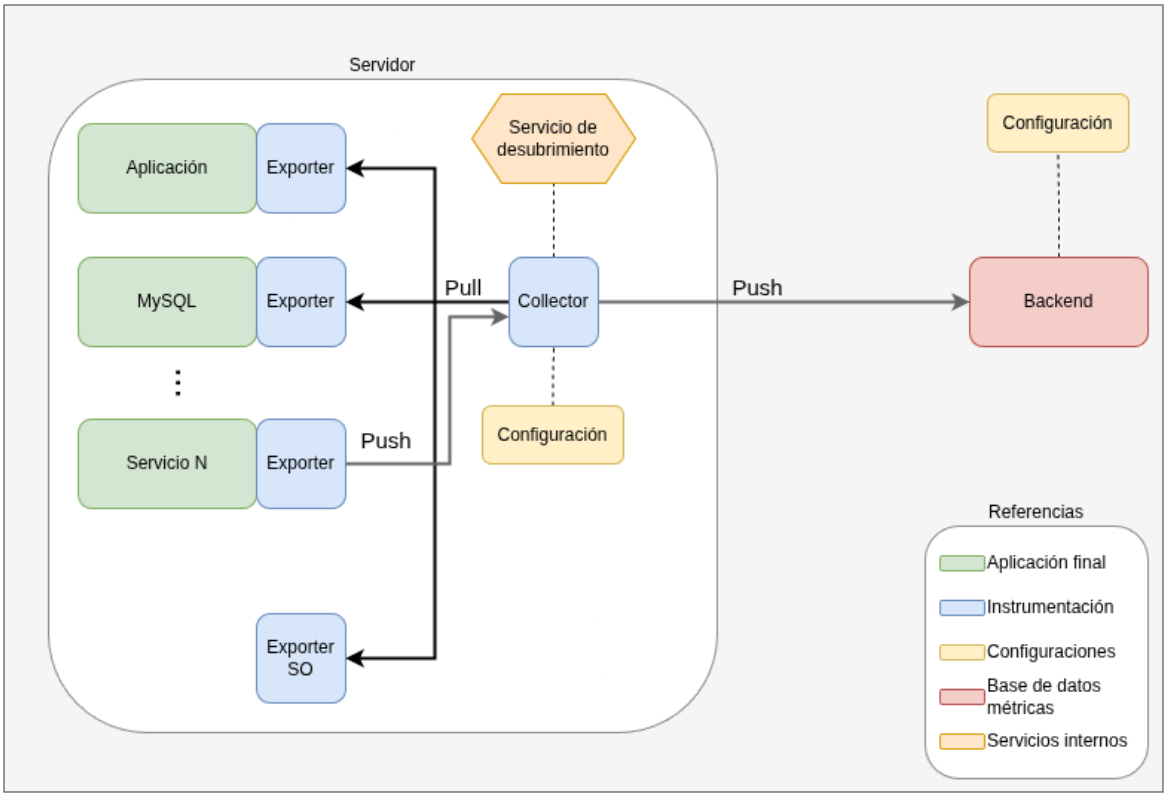
tipo de arquitecturas deja el grueso de la lógica al servidor central, mientras que los agentes suelen ser bastantes más livianos, ya que solo se encargan de recuperar y disponibilizar los datos para que el servidor central los recupere. Uno de los principales exponentes de esta arquitectura es el stack propuesto por Prometheus, compuesto por el servidor de métricas homónimo, Alertmanager, Grafana y una gran variedad de agentes.



Con la estandarización de productos como Opentelemetry, aparece un nuevo concepto, llamado colector. El mismo cumple varias funciones, entre las más destacadas se encuentra

- Recuperar datos directamente del servidor en el que se encuentra instalado, esto
- Exponer servicios de recolección de datos usando protocolos conocidos, como puede ser OTLP, el protocolo desarrollado por Opentelemetry

Esto nos dejaría una arquitectura como la siguiente



Problemáticas

Al momento de configurar y orquestar todos estos productos surge una de las principales problemáticas que nos vamos a encontrar muy a menudo, no todos los productos están preparados para ser observados, y en muchos casos terminamos cayendo en simplemente monitorear el estado de algunos recursos en forma de métricas, y recuperando logs, sin correlacionarse, y sin enriquecer nuestra visión sobre el estado interno de la aplicación. Para esto es necesario que, en principio, podamos satisfacer los tres pilares que ya vimos de la observabilidad, para luego poder empezar a generar información con nuestros datos. A lo largo de esta investigación, presentaremos cuáles son los puntos a analizar en el desarrollo de aplicaciones para poder mejorar su disponibilidad.

Prácticas recomendadas

A lo largo del ciclo de vida de un desarrollo, y su posterior puesta en producción, vamos a poder identificar diferentes prácticas recomendadas que nos van a facilitar las tareas, y van a permitir retroalimentar las diferentes etapas para mejorar así tanto la calidad del software, como del servicio brindado. A lo largo de este capítulo, vamos a recorrer tres grandes etapas, el desarrollo, el despliegue y las operaciones, y en cada etapa vamos a recopilar y explicar dichas prácticas.

Desarrollo

El desarrollo de aplicaciones cloud-native implica adoptar un conjunto de prácticas y principios que trascienden la escritura de código e impactan de manera transversal en la arquitectura, los procesos de trabajo y la operación del sistema. En esta sección se abordan tres pilares fundamentales que estructuran dicho enfoque.

En primer lugar, se analiza la metodología Twelve-Factor App, formulada en 2012 por ingenieros de Heroku, empresa referente en la provisión de servicios cloud. Esta metodología establece doce principios que permiten evaluar y guiar el diseño de una aplicación orientada a entornos cloud, abarcando desde aspectos básicos de configuración y gestión de dependencias hasta consideraciones de mayor complejidad que pueden requerir cambios profundos en la forma de trabajo del equipo.

En segundo lugar, se profundiza en el concepto de integración continua (Continuous Integration, CI), considerándolo un elemento central en el desarrollo de aplicaciones que deben crecer de forma ágil y sostenida. La integración continua, junto con la entrega y el despliegue continuos (CD), conforma un conjunto de prácticas que permiten acortar los

ciclos de desarrollo, reducir el riesgo de errores y mantener el software en un estado consistentemente desplegable.
Por último, se examina la observabilidad como disciplina operacional, recorriendo sus pilares fundamentales (métricas, trazas distribuidas y registros) e identificando las prácticas clave que permiten comprender el comportamiento interno de un sistema a partir de sus señales externas.

12 factor apps

Los Twelve-Factor App[25] constituyen un conjunto de principios y prácticas recomendadas para el desarrollo de aplicaciones modernas y escalables en entornos cloud-native. Formulados por un grupo de desarrolladores de Heroku y documentados por Adam Wiggins en 2012, estos factores se han consolidado como estándares de referencia para el desarrollo de aplicaciones basadas en arquitecturas de microservicios, y se alinean con los principios del diseño de software ágil. A continuación se analiza cada uno de ellos:

1. *Base de código única*: Se debe mantener una base de código única en un repositorio de control de versiones, de modo que todas las instancias de la aplicación se deriven de esa misma fuente. En escenarios de sistemas distribuidos, cada servicio debería contar con su propio repositorio, evitando así el acoplamiento entre componentes. A partir de ese repositorio único se generan todos los despliegues posibles, ya sea hacia entornos de producción y staging como hacia los entornos locales de cada desarrollador, cuya importancia se abordará más adelante.
2. *Dependencias explícitas*: Las dependencias del sistema deben declararse de forma explícita y aislarse mediante administradores de paquetes con versiones específicas. Se debe evitar la dependencia de herramientas o bibliotecas instaladas globalmente en el sistema operativo, priorizando en cambio que todas las dependencias sean declaradas y resueltas en el ámbito del propio repositorio.
3. *Configuración en tiempo de ejecución*: La configuración de la aplicación debe almacenarse en variables de entorno, lo que permite modificarla sin alterar el código fuente y aporta dinamismo a los distintos ambientes. Adicionalmente, el uso de variables de entorno hace que la configuración sea agnóstica tanto al sistema operativo como al lenguaje de programación utilizado.
4. *Servicios de soporte como recursos adjuntos*: Los servicios externos, como son bases de datos, colas de mensajes, servicios de caché, entre otros, deben tratarse como recursos que pueden conectarse y desconectarse sin requerir cambios en el código. Una aplicación que sigue este principio no distingue entre un servicio local y

uno provisto por un tercero: debe ser posible, por ejemplo, sustituir una instancia local de MySQL por un servicio administrado como Amazon RDS sin modificar el código fuente.

5. *Construcción, despliegue y ejecución como etapas separadas*: Las etapas de construcción, despliegue y ejecución deben estar claramente delimitadas y ser independientes entre sí. La etapa de construcción debe automatizarse y ejecutarse ante cada integración a la rama principal, según el esquema de ramificación adoptado; en ella es fundamental fijar de forma explícita la versión generada, la cual debe ser única e irrepetible. El despliegue, por su parte, puede ser manual o automatizado, aunque este último caso requiere implementar un flujo de entrega continua, aspecto que se desarrollará en la siguiente sección.
6. *Procesos sin estado*: Fuera del entorno de desarrollo, la aplicación debe ejecutarse como uno o varios procesos sin estado. Cualquier dato que deba persistirse ha de delegarse a un servicio externo, conforme a lo establecido en el factor anterior. Este principio es condición necesaria para el escalamiento horizontal, ya que permite agregar o remover instancias de la aplicación sin afectar su comportamiento.
7. *Asignación de puertos*: La aplicación debe ser autocontenida y exponer sus servicios a través de un puerto específico en el que atiende las peticiones. Este principio refuerza lo planteado en el factor 4, ya que permite que la propia aplicación actúe como servicio de soporte para otras, sin depender de servidores web externos ni de configuraciones de infraestructura adicionales.
8. *Concurrencia mediante procesos*: La aplicación debe diseñarse para aprovechar el paralelismo escalando horizontalmente mediante múltiples procesos ligeros, en lugar de depender de procesos monolíticos de gran tamaño. Este modelo no es incompatible con lenguajes que ofrecen soporte nativo para multithreading, como Java, o para operaciones asíncronas, como Node.js. Por el contrario, se plantea que dichas capacidades deben complementarse con la posibilidad de desplegar múltiples procesos sin estado en paralelo.
9. *Descartabilidad*: La aplicación debe diseñarse para iniciar con rapidez, lo que facilita el escalamiento horizontal, y para gestionar de forma segura una terminación en cualquier momento. Esta capacidad de inicialización rápida y apagado ordenado incrementa la robustez del sistema frente a fallas o reinicios inesperados.
10. *Paridad entre entornos*:Los entornos de desarrollo, pruebas y producción deben mantenerse lo más similares posible en cuanto a infraestructura, dependencias y configuración. Reducir las diferencias entre entornos minimiza los problemas de compatibilidad y los comportamientos inesperados que suelen surgir cuando el código se ejecuta en condiciones distintas a las de desarrollo.

11. *Registros como flujos de eventos*: Los registros o logs de la aplicación deben tratarse como flujos de datos continuos. La aplicación no debe preocuparse por su enrutamiento o almacenamiento; simplemente debe escribir sus eventos en la salida estándar, dejando que la infraestructura se encargue de capturarlos, agregarlos y analizarlos mediante las herramientas correspondientes.
12. *Procesos administrativos como tareas puntuales*: Las tareas administrativas, como migraciones de base de datos, ejecución de scripts de mantenimiento, consolas interactivas, etc, deben ejecutarse como procesos independientes y de una sola ejecución, separados de la aplicación principal. Esto garantiza que dichas operaciones no interfieran con el ciclo de vida normal de la aplicación.

Integración contínua

Desarrollaremos las bases para definir un flujo de integración contínua que nos permita maximizar la eficiencia y calidad durante el desarrollo de software.

- Automatización de la compilación
- Pruebas automatizadas
- Esquema de ramificación
- Gestión de dependencias
- Entornos aislados y replicables
- Monitorización y registros
- Revisión de código

Un punto que es importante destacar, es que además de pensar la observabilidad de nuestras aplicaciones, podemos disponibilizar métricas del proceso de CI. Cuando se analizan los datos en esta instancia, surgen varios indicadores que ayudan a mejorar los procesos internos, detectar retrasos y visualizar la eficiencia del equipo.

Automatización de la compilación

La automatización de la compilación es un componente clave en la integración continua, ya que permite asegurar que el código fuente se compila de manera consistente y confiable en cada integración. Un punto importante para asegurar la correcta implementación de este proceso es la estandarización del ambiente en el que se va a compilar. Para esto, se

recomienda utilizar IaC y productos que permitan versionar la configuración de sus pipelines.

Pruebas automatizadas

Ejecutar la batería de pruebas de forma automática en las integraciones de nuestro proyecto va a garantizar que la calidad del software no se vea afectada en la medida que el proyecto crece. Dentro de los tipos de pruebas, podemos incluir pruebas unitarias, pruebas de integración, pruebas funcionales, y pruebas punto a punto.

- *Pruebas unitarias*: Son aquellas pruebas que buscan validar el correcto funcionamiento de funciones o métodos dentro de nuestro código.
- *Pruebas de integración*: Son aquellas pruebas que buscan validar el funcionamiento de varios módulos o servicios en conjunto. Para este tipo de pruebas se puede necesitar el soporte de servicios externos como puede ser una base de datos o un servicio de cola.
- *Pruebas funcionales*: Son aquellas pruebas que buscan validar la lógica de negocio de la aplicación. Para esto, en muchos casos se va a requerir que se levanten múltiples componentes o servicios externos como es el caso de las pruebas de integración, lo que lo diferencia de éstas últimas es el tipo de verificación que se busca en cada caso, mientras que en las pruebas de integración se busca validar el correcto funcionamiento de las comunicaciones entre los diferentes módulos o servicios, en las pruebas funcionales se busca, además, validar que la respuesta es correcta basándose en la propia lógica del negocio.
- *Pruebas punto a punto*: También llamadas pruebas e2e, del inglés, end-to-end. Son aquellas pruebas que buscan replicar el comportamiento de un usuario en la aplicación. Son pruebas muy costosas y complejas de implementar, ya que requieren replicar el ambiente completo de la aplicación, y además se debe replicar el comportamiento de forma que se pueda ejecutar automáticamente. Además, este tipo de pruebas suelen ser difíciles de mantener actualizadas en el tiempo ya que suelen estar muy acopladas a las versiones del software, ya sea por la interfaz o por las comunicaciones entre los servicios.

Revisión de código

Generar revisiones de código permite mejorar en gran medida tanto la calidad del código, generando un producto mucho más robusto, como también mejorar las interacciones del

equipo, ya que es necesaria una comunicación fluida para que esta metodología funcione correctamente. Por otro lado, permite alivianar la carga a los nuevos integrantes que se suman al equipo, ya que sus cambios van a estar siendo validados por sus propios compañeros.

Esquema de ramificación

Definir el esquema de ramificación correcto va a requerir conocer el esquema de versionado y soporte que se quiere respetar, el tamaño del equipo de desarrollo. Entre los esquemas más conocidos, podemos identificar

- *Gitflow*: Es un esquema muy estructurado en donde vamos a contar con diferentes tipos de ramas, las cuales tienen asignadas una serie de procesos para poder interactuar entre las mismas para poder integrar de forma segura. Podemos identificar 5 tipos de ramas diferentes, la rama principal, normalmente llamada “main”, es en donde tenemos la última versión funcional del proyecto, junto a ésta, se encuentra la rama de integración, que va a ser nuestra rama principal de desarrollo, normalmente la llamamos “development” o “develop”, es desde esta rama que van a partir la gran mayoría de las ramas de nuestro proyecto. Luego tenemos las ramas de funcionalidad, normalmente llamadas usando el prefijo “feature/”, es en estas ramas que se va a trabajar en nuevas funcionalidades, normalmente por tiempos reducidos a no más de unos días. Parte desde la rama de integración, y al finalizar el desarrollo de la funcionalidad se junta nuevamente con la misma rama. Después tenemos las ramas de lanzamiento, normalmente llamadas usando el prefijo “release/”, usadas con el fin de terminar de modificar nuestro proyecto para sacar una nueva versión, entre las tareas que se pueden identificar, se encuentra la modificación del archivo de versión del proyecto, actualizar registros de cambios (del inglés, changelog), actualizar documentaciones, o hacer arreglos de último momento. Estas ramas también se crean a partir de nuestra rama de integración, aunque se terminan juntando con la rama principal, además de juntarlo con la rama de integración. Finalmente tenemos las ramas de arreglos, normalmente llamadas usando el prefijo “hotfix/”, es desde donde se solucionan problemas con versiones productivas. Es la única rama que se crea a partir de la rama principal, y se terminan juntando tanto en la rama principal como en la rama de integración.
- *Github flow*: Es un esquema simplificado que permite el crecimiento orgánico de los proyectos de forma ordenada, teniendo como principal premisa la integración contínua. En comparación con *Gitflow*, si bien vamos a tener ramas con roles

específicos, solo vamos a contar con dos tipos, la rama de integración es a su vez la rama principal, que es donde en todo momento deberíamos encontrar el código estable. A su vez esta rama principal, que se suele llamar “main”, debe estar en condiciones de ser desplegada en ambientes productivos. Por otro lado todo cambio que se quiera hacer en la rama principal se hace mediante el segundo tipo de rama, la cual se crea a partir de la rama principal, y no debe extenderse más que unos pocos días. La regularidad con la cual se debe integrar va a depender en muchos casos del propio equipo y del tipo de funcionalidad que se esté trabajando. Al momento de integrar, se debe hacer mediante un “Pull Request” de Github, en el cual vamos a poder aplicar buenas prácticas tales como

  - *Revisión de código*: Explicado anteriormente, permite mejorar tanto la calidad del código, como fomentar la comunicación interna del equipo.
  - *Ejecución de batería de pruebas*: Explicado anteriormente, nos va a permitir asegurarnos que la nueva funcionalidad se integra de forma correcta al código existente, sin provocar efectos no deseados.
  - *Generación de documentación*: Una buena estrategia es documentar el código de forma que posteriormente podamos usar herramientas tales como es el caso de pydoc o sphinx en python, go doc, en golang, o yard en ruby..
  - *Entrega contínua*: Necesario para el siguiente punto, generar el o los artefactos de nuestro proyecto de forma automática en cada Pull Request nos facilita la distribución de los mismos, ya sea para pruebas en ambientes automáticos, como pruebas locales por parte del equipo de desarrollo o equipo de QA.
  - *Generación de ambientes de pruebas*: Le va a permitir tanto al grupo de desarrollo, como a un posible grupo de QA o grupo selecto de clientes finales, probar las nuevas funcionalidades en la medida que se desarrollan. Esto, junto a la ejecución de batería de pruebas permite reducir en gran medida efectos no deseados sobre el proyecto.

- *Gitlab flow*: Es un esquema con dos posibles flujos, cuya finalidad es la de proveer un proceso para dos casos conocidos en proyectos, uno en el que solamente se brinda soporte para una sola versión, este caso además es el resuelto por *Git flow*, mientras que el segundo caso es aquel en el que tenemos que brindar soporte para múltiples versiones a lo largo del tiempo, este caso es común en productos de software libre, y sobre todo en librerías. En el primer caso vamos a tener una rama principal en la cual vamos a ir integrando las nuevas funcionalidades mediante nuevas ramas y un esquema similar al planteado en *Github flow*, mediante MergeRequest, y vamos a contar ahora con una rama llamada “production”, la cual

debe reflejar en todo momento el código que se encuentra desplegado en el ambiente productivo, se recomienda que se haga mediante despliegue contínuo al momento de integrar “main” a “production”. Por otro lado, tiene soporte para múltiples ambientes, definiendo una rama para cada ambiente que se quiera implementar, la forma de trabajo va a ser igual que con la rama productiva, y ante cada commit vamos a lanzar la acción de desplegar automáticamente en el ambiente que corresponda. Al utilizar varios ambientes, se va a armar una especie de jerarquía, en donde las integraciones se inician desde los ambientes más alejados a producción, como por ejemplo, “testing”, y se pasa por todos los ambientes hasta llegar a producción.

En el segundo caso, se plantea un flujo también basado en ramas, en donde cada par de versiones x, y siguiendo el versionado semántico, va a contar con su rama. Por ejemplo, vamos a tener las ramas “2.4-stable” y “2.5-stable”. Todo lo que sean arreglos comunes, se van a trabajar en una rama nueva y se van a integrar en las diferentes ramas mediante la operación de git “cherry-pick”.

- *Trunk Based Development*: Es un modelo de ramificación con foco en la integración contínua, el cual cuenta con dos posibles implementaciones. La primera está destinada a equipos pequeños, o preferentemente pares de programadores que pueden implementar pair-programming, y se basa en tener una única rama principal, llamada “Trunk”, la cual va a ser la única rama a la cual se van a subir cambios, estos cambios deben pasar, al menos, una etapa de construcción previa de forma obligatoria para evitar la integración de código no funcional. Luego se tiene diferentes ramas que representan las diferentes versiones que se van a usar como solo lectura. Esto permite organizar de una forma muy estructurada la historia de versiones, mientras suprime la posibilidad de conflictos en las integraciones, ya que no cuenta con este tipo de operaciones.

  La segunda implementación ya está pensada para grupos medianos y grandes, y se basa en seguir teniendo como rama compartida, la rama “Trunk”, pero que se va a modificar mediante el esquema de Pull Request o Merge Request, dependiendo de la herramienta en la que estemos trabajando. Estos Pull Request, se van a aplicar usando sub-ramas que se crean a partir de la rama principal, y que no deberían existir por más de unos pocos días. Otro punto importante en este enfoque es la posibilidad de automatizar tareas en el estadío de Pull Request, como puede ser la ejecución de pruebas, ya sea de unidad, como de integración, funcionales o e2e, como también la implementación de mecanismos de revisión de código. En este esquema se sigue utilizando las ramas para alojar las versiones, siendo también de solo lectura.

Gestión de dependencias

Como ya vimos, toda aplicación que siga los 12 factores, ya debería usar un gestor de dependencias. En el caso que no sea así, es de suma importancia hacerlo para poder asegurar la correcta construcción de los artefactos. Contar con nuestras dependencias versionadas, junto a un archivo de lock que permite identificar de forma unívoca la versión de una dependencia nos permite construir un proceso de integración predecible y replicable.

Entornos aislados y replicables

El entorno en el que se ejecutan cada una de las tareas de integración debe encontrarse aislado del entorno en el que se ejecutan los pipelines, por ejemplo, usando contenedores. A su vez, construir nuestros pipelines en un entorno basado en contenedores nos permite replicar cada uno de los ambientes necesarios de forma simple.

Observabilidad

Al iniciar un nuevo proyecto de desarrollo, es fundamental identificar los aspectos clave que se deben considerar para una gestión efectiva del servicio. Estos puntos pueden ser de naturaleza general, como el uso de recursos, pero también deben abarcar particularidades del negocio, como por ejemplo el estado de una transacción o la saturación de recursos críticos (como un servicio de colas). Es crucial tener esto en cuenta durante la planificación y el desarrollo, ya que facilitará la exportación de estos datos en el formato más adecuado, ya sean métricas, registros o trazas.
Reconociendo que no todos los proyectos pueden implementar un análisis exhaustivo, a continuación se detallarán algunos de los puntos más relevantes para optimizar la observabilidad, clasificándolos, según los tres pilares fundamentales.

Métricas

Las métricas de uso de recursos se pueden obtener directamente de la infraestructura subyacente, eliminando la necesidad de desplegar agentes específicos en nuestro producto. Ejemplos de servicios que proveen estas métricas incluyen cadvisor[35] para contenedores, y node exporter[35] o process exporter[36] para servicios en distribuciones Linux.
En el contexto de un servicio web, se puede acceder a métricas relevantes desde el *application server*, el *web server*, y el *load balancer*. Además, en entornos basados en

contenedores, el *driver de red* puede ser una fuente de métricas. Una aproximación más general y avanzada es el uso de un *service mesh*[3] o su evolución, el *ambient mesh*[37]. Estos últimos ofrecen herramientas adicionales para la depuración de las comunicaciones internas.

Logs

Para garantizar la observabilidad, es fundamental considerar tres aspectos clave en la configuración de los registros (*logs*):

1. Nivel de Verbosidad: este parámetro es crucial, ya que determina la cantidad de información sobre el estado de las operaciones que reporta la aplicación. Un nivel bajo puede llevar a una escasez o nulidad de datos útiles, mientras que un nivel excesivamente alto podría degradar la *performance*. La configuración ideal debe ser adaptada y personalizada para cada servicio y ambiente.
2. Destino de los Registros: Siguiendo la metodología twelve-factor, los registros deben dirigirse siempre a la salida estándar y la salida de error estándar. Esto simplifica su recuperación, especialmente al trabajar en entornos de contenedores.
3. Formato Estructurado: La adopción de un formato estructurado y estándar, como JSON, facilita considerablemente el procesamiento y la recuperación de información de los registros.

Trazas

En lo que respecta a la gestión de trazas, es recomendable instrumentar soluciones de observabilidad que proporcionan visibilidad completa del comportamiento de las aplicaciones y componentes de sistemas distribuidos. En este contexto, se recomienda la adopción de librerías y estándares que faciliten la recolección, el procesamiento y la exportación de datos de telemetría.

Como ya hemos mencionado en secciones anteriores, OpenTelemetry emerge como una solución y un estándar de facto en la industria. Esta iniciativa open source ofrece un conjunto robusto de herramientas, APIs y SDKs que permiten instrumentar código de forma uniforme, independientemente del lenguaje de programación. Las ventajas destacables de implementar OpenTelemetry incluyen:

- Portabilidad y Estandarización: Define un formato de datos de telemetría (trazas, métricas y *logs*) neutral respecto al *vendor*, lo que evita el *vendor lock-in* y permite integrar fácilmente diferentes backends de análisis como Jaeger, Zipkin,

Prometheus, o soluciones comerciales.

- Auto-instrumentación: Uno de los mayores atractivos de OTel son los *auto-instrumentadores*. Estas herramientas permiten a los desarrolladores obtener trazas distribuidas sin necesidad de modificar el código fuente de la aplicación, simplemente ejecutando la aplicación con las librerías o agentes apropiados. No obstante, es crucial señalar que el nivel de soporte y la madurez de los auto-instrumentadores varían entre los lenguajes de programación, por ejemplo, lenguajes como Java o Python suelen tener mejor soporte que otros.

Monitoreo de caja negra

En la monitorización de sistemas informáticos, al igual que en las pruebas de desarrollo, se distinguen las metodologías de caja blanca y caja negra.
La monitorización de caja blanca implica tener acceso al servicio o a la infraestructura de despliegue para la obtención de datos. Lo presentado hasta el momento se enmarca en estas prácticas, que permiten comprender las causas del mal funcionamiento de un servicio.
Por otro lado, la monitorización de caja negra se basa en la ejecución de pruebas sintéticas que simulan el comportamiento de los sistemas desde la perspectiva del usuario final. A diferencia de la caja blanca, esta metodología solo indica si el sistema responde o no según lo esperado, sin detallar las razones de un fallo. Estas pruebas no se limitan a HTTP, pudiendo incluir protocolos como ICMP, TCP, DNS o gRPC.
Adicionalmente, se recomienda distribuir geográficamente los puntos de prueba para obtener valores que reflejen mejor la experiencia de usuarios en distintas ubicaciones. Por ejemplo, se podrían realizar estas pruebas desde Argentina y Uruguay para un sistema con uso regional.

Despliegue y operaciones

En esta sección vamos a desarrollar la etapa siguiente de la integración contínua, que culmina en el despliegue automático de software. Partiendo de un proceso sólido de integración contínua (CI) que garantiza la calidad del código, la ejecución automatizada de pruebas y la generación consistente de artefactos o entregables, el siguiente paso lógico es la Entrega Contínua o CD del inglés, Continuous Delivery. Este proceso asegura que el software esté siempre listo para ser liberado a producción con solo presionar un botón, o bien, de forma completamente automatizada.

Para maximizar el valor de la Entrega Contínua, es deseable avanzar hacia el Despliegue Contínuo o CD del inglés, Continuous Deployment (CD puede confundir despliegue continuo respecto de entrega continua dado que en inglés las siglas son iguales). Esta práctica va un paso más allá, dado que automatiza la publicación de cada nuevo release que pasa exitosamente todas las etapas de uno o varios pipelines de CI/CD, directamente a un ambiente de producción o similar. Es decir, cada *commit* que pasa las pruebas puede terminar automáticamente siendo liberado a los usuarios, sin intervención humana.

Lograr un Despliegue Contínuo exitoso y seguro, especialmente cuando se realiza de forma desatendida, requiere la consideración e implementación de los siguientes pilares clave, enfocados en maximizar la calidad, la estabilidad y la confiabilidad del proceso:

1. Ambientes de despliegue: Es fundamental contar con ambientes (desarrollo, staging, producción) que sean lo más parecidos posible. Esto reduce los errores que solo se manifiestan en producción. La definición de estos ambientes debe estar versionada y automatizada.
2. Estrategias de despliegue: Para publicar un nuevo release de forma desatendida minimizando el riesgo de interrupción del servicio, se deben utilizar estrategias que permitan introducir el cambio gradualmente.
3. Automatización: La gestión y el despliegue de la infraestructura, los ambientes y la aplicación deben estar completamente automatizados y definidos como código, por ejemplo, con Terraform, Ansible, Pulumi, etc. El enfoque GitOps extiende esto al usar Git como la única fuente de verdad declarativa para la infraestructura y las aplicaciones, donde cualquier cambio en el repositorio de Git resulta automáticamente en un cambio en el ambiente o aplicación.
4. Monitoreo y Registros: El proceso de Despliegue Contínuo debe estar estrechamente ligado a sistemas robustos de monitoreo. Esto implica:
    a. Monitoreo del Proceso (CI/CD): Seguimiento del tiempo de ejecución y el estado de cada etapa de los pipelines.
    b. Monitoreo de la Aplicación: Recolección de métricas clave como latencia, tasa de errores, saturación, uso de recursos de las aplicaciones.
    c. Registros Centralizados: Todos los *logs* de las aplicaciones y el de los componentes de infraestructura deberían centralizarse y quedar accesibles para un diagnóstico rápido, permitiendo la detección temprana de anomalías post-despliegue.
5. Rollbacks Automatizados y Confiables: En un proceso de despliegue desatendido, la capacidad de volver automáticamente a un estado operativo previo es esencial si el monitoreo detecta que el nuevo release está fallando. Un rollback debe ser rápido y activarse automáticamente, o con un solo comando manual de emergencia, si se

superan umbrales de error o latencia previamente definidos. La infraestructura y las bases de datos deben estar diseñadas para soportar la reversión de cambios de forma segura.

Ambientes de despliegue

Para asegurar un despliegue continuo seguro y sostenible, es esencial establecer un esquema basado en las validaciones necesarias del producto. Habitualmente, se definen ambientes clave como Desarrollo, QA, Staging y Producción. La gestión y generación de estos entornos es crítica, priorizando la paridad entre ellos para prevenir desajustes al promover un despliegue de un entorno a otro. Esto requiere enfocarse principalmente en:

- Sistema operativo.
- Paquetes instalados en el sistema operativo.

Además, es altamente recomendable que el hardware sea lo más similar posible, lo que facilita una estimación precisa del rendimiento.

Esta tarea se simplifica considerablemente al utilizar métodos automatizados y desatendidos para el aprovisionamiento de sistemas operativos y la preparación de entornos, eliminando el riesgo de errores humanos. Usando contenedores, esta tarea se simplifica aún más.

Estratégia de despliegue

Una estrategia de despliegue que garantice la validación efectiva en diferentes ambientes antes de llegar a producción, y que además permita volver a una versión anterior ante cualquier inconveniente, es fundamental para mejorar la calidad del proceso de despliegue. Entre las estrategias más reconocidas se encuentran:

- Canary deployment: Es una estrategia en la que se cuenta con un ambiente replicado en el cual se va a desplegar la nueva versión, para posteriormente redirigir solamente una porción del tráfico con el fin de comprobar el correcto funcionamiento de la nueva versión. Esta porción de tráfico representa un conjunto de usuarios destinado a probar las nuevas funcionalidades. Esta estrategia suele usarse por probadores de funcionalidedes beta, o también seleccionando solamente un porcentaje del tráfico normal del producto que se redirige. En este punto, es de suma importancia contar con observabilidad, ya que proporcionará información importante que determine el correcto funcionamiento, y de esta forma determinar si realizar un rollback o avanzar con un porcentaje mayor. Esta estrategia tiene como beneficio

que, al utilizar un grupo de instancias acotado sólo consumen recursos durante el momento de actualización, por lo que es una buena alternativa ante el escenario de recursos limitados.

- Blue-Green deployment: Es una estrategia en la que se cuenta con un ambiente replicado en los cuales se van a desplegar las nuevas versiones. Para su implementación se debe contar con algún balanceador de tráfico que permita seleccionar cuál de los ambientes, Blue o Green, es el que actualmente funciona como ambiente productivo, y cuál como ambiente de pre-producción. De igual forma que la estrategia anterior, este esquema puede ser potenciado con el uso de la observabilidad para brindar información del funcionamiento y agregar la posibilidad de realizar un rollback ante cualquier problema, siendo en este caso, únicamente el cambio en el balanceador. Esta estrategia tiene como desventaja, el alto consumo de recursos, ya que se debe contar con un ambiente espejado en todo momento.
- Rolling update: Es una estrategía en la que se cuenta solamente con un único entorno, el cual debe tener a disposición recursos disponibles para escalar horizontalmente. En este enfoque se actualizan lotes de las versiones de las diferentes réplicas. Cada lote se toma como un porcentaje de las réplicas antes de comenzar. El primer paso, creará un lote de N réplicas con la nueva release. Cuando estén todas funcionales, apagará un lote de N versiones anteriores. El proceso se repite hasta que todas las réplicas se correspondan con la nueva versión. También en esa estrategia, se necesita contar con un balanceador de carga que permita distribuir el tráfico entre las diferentes instancias.

Infraestructura como código

La implementación de Infrastructure as Code (IaC) es esencial para reducir significativamente los tiempos de despliegue, dado que permite automatizar la creación y configuración de ambientes.

Este proceso se lleva a cabo mediante el uso de herramientas especializadas como, ansible, saltstack, puppet, terraform, etc que facilitan la creación, configuración y gestión eficiente de la infraestructura requerida. Los contenedores son también una forma más simple de armar ambientes autocontenidos para cada aplicación, lo cuál elimina el uso de herramientas de IaC en favor del armado de Containerfiles o Dockerfiles que utilizan simples comandos del shell.

Además, es importante utilizar un Sistema de Control de Versiones (SCV). Esta práctica

aporta beneficios fundamentales como:

- Asegurar la consistencia y replicabilidad de los entornos.
- Garantizar la trazabilidad de cada modificación, aprovechando la naturaleza intrínseca del SCV.
- Permitir la integración de buenas prácticas y estándares propios del desarrollo de software, incluyendo la revisión de código o Code Review, y el uso de análisis estático para reforzar los estándares de codificación.

Observabilidad y los despliegues

Como ya mencionamos, la observabilidad no se limita únicamente a prestar especial atención a lo que sucede dentro de nuestras aplicaciones. Una perspectiva crucial es la integración de los datos que surgen directamente del proceso de despliegue. Unificando la información de la aplicación en tiempo real, junto a eventos y estados propios de CI/CD, es lo que realmente ofrece una perspectiva real, completa y operativa de qué pasa en el ecosistema luego de poner una nueva versión o un cambio en producción.
El ciclo de vida del software, desde el commit inicial hasta el despliegue final en producción, es complejo. Al incorporar la observabilidad en las etapas de Integración Continua (CI) y Entrega Continua (CD), estamos dotando al proceso de una capa vital de feedback rápido y preciso.
Utilizar herramientas de logs, métricas y trazas para detectar problemas de rendimiento, errores, regresiones o comportamientos inesperados durante las etapas de CI/CD puede generar un circuito de retroalimentación valioso a los equipos de desarrollo.
En esencia, la observabilidad en los despliegues transforma la pregunta: *"¿Está funcionando la aplicación?"* en una mucho más poderosa: *"¿El cambio que acabo de realizar está teniendo el impacto deseado y esperado en la experiencia de usuario y la salud del sistema?".*

Rollbacks automatizados

Se debe establecer un mecanismo automatizado para revertir rápidamente un despliegue en caso de que se detecten problemas. Esto requiere una estrategia de respaldo definida y un método controlado para deshacer los cambios implementados. La elección del mecanismo de *rollback* a menudo está ligada a la estrategia de despliegue seleccionada.

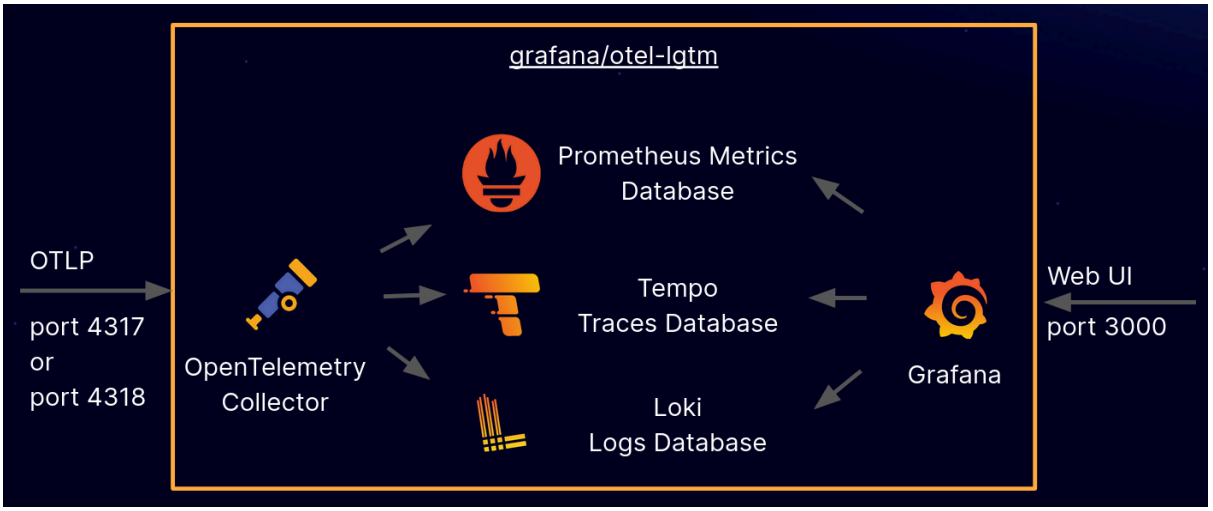
Es importante entonces considerar el proceso a seguir en caso de que se hayan aplicado migraciones a la base de datos. Sin embargo, dado que estos procesos específicos están

fuera del alcance de la presente tesis, no se profundizará en los posibles enfoques para su implementación.

Arquitecturas de observabilidad

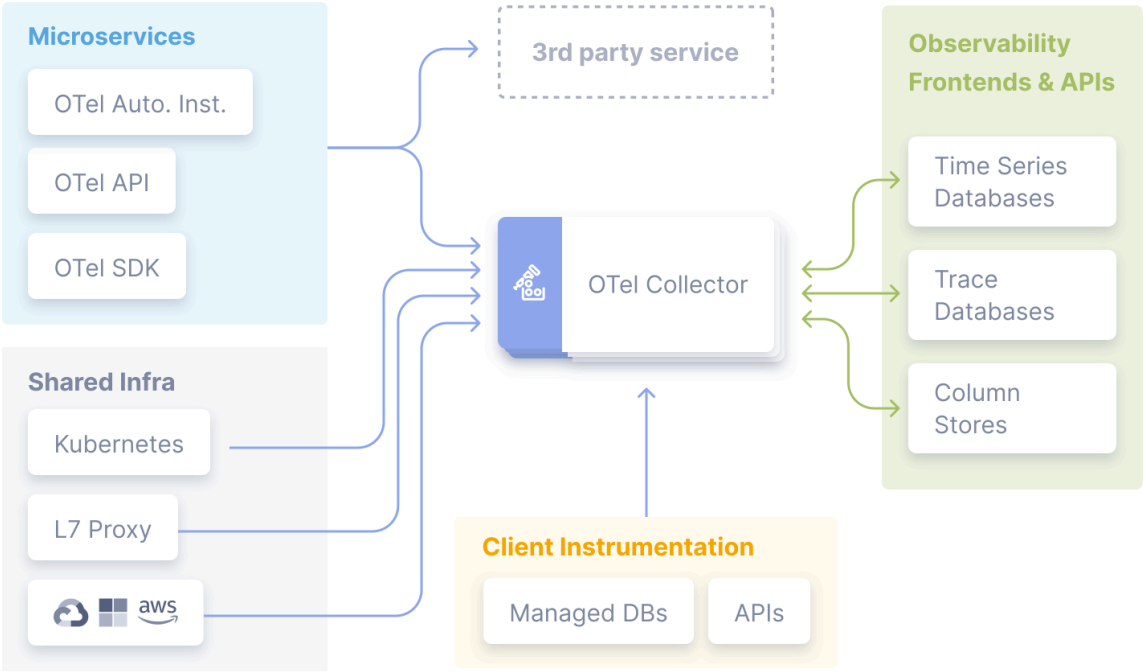
Actualmente, la tendencia en arquitecturas de observabilidad se orienta al desacoplamiento de las bases de datos de series de tiempo respecto a los componentes de recolección y transformación de datos. Esta evolución se hace evidente con la adopción de OpenTelemetry como elemento central en diversas arquitecturas, incluyendo entornos cloud, máquinas virtuales, orquestadores de contenedores como Kubernetes y serverless. Incluso servicios líderes del mercado como GrafanaLabs, que añadió soporte para el protocolo OTLP en noviembre de 2023[38], New Relic o incluso Datadog se están adaptando a este estándar.
En el flujo de datos actual, se observan arquitecturas híbridas, donde agentes que recuperan datos en modalidad pull que luego envían a recolectores en modalidad push. Estos recolectores se encargan de la transformación de los datos antes de su almacenamiento. De esta forma, las arquitecturas de observabilidad modernas se caracterizan por el desacoplamiento de las funciones de recolección y transformación. Esto se logra mediante la articulación de diferentes servicios y una especial atención a la estructuración de la información para permitir la correcta correlación de los tres pilares de la observabilidad. Esto proporciona un valor añadido, superando la limitación de herramientas previas que solo permitían recuperar métricas o registros de forma aislada. La capacidad de establecer "puentes" entre los datos de los diferentes pilares facilita procesos de depuración más complejos y eficientes, así como análisis mucho más completos.

En nuestro caso de estudio, adoptaremos la arquitectura propuesta por Grafana, conocida como **LGTM**, que integra sus productos: **Loki, Grafana, Tempo y Mimir**.



Fuente: documentación grafana.com

Se puede notar que el colector de OpenTelemetry funciona para todas las bases de datos, comunicándose mediante un protocolo estándar (OTLP). Esto permite desacoplar de forma simple la generación de datos y recolección de los datos, del almacenamiento. A su vez, cada colector puede funcionar a modo de agregador local, agregando una capa escalable de buffer, que evite pérdidas de información ante la no disponibilidad de las bases de datos.



Aplicación

Para las pruebas se seleccionaron tres productos desarrollados en lenguajes y frameworks diferentes, con el fin de poder identificar y comparar las distintas formas de configurar la instrumentación con OpenTelemetry en aplicaciones que no fueron diseñadas originalmente con observabilidad en mente. Los casos de estudio seleccionados fueron:

- Wordpress: un CMS desarrollado en PHP, publicado en el año 2003.
- Redmine: una herramienta de gestión de proyectos y tickets desarrollada en Ruby, usando el framework Ruby on Rails, publicada en el año 2006.
- Wagtail: un CMS desarrollado en Python, usando el framework Django, publicado en el año 2014.

Es importante destacar que ninguno de estos productos fue diseñado teniendo en cuenta los 12 factores descritos anteriormente, ni aplicando técnicas orientadas a facilitar la observabilidad de sus componentes. Esto los convierte en casos de estudio representativos del software preexistente que típicamente se encuentra en entornos productivos.

Instrumentación

Para garantizar una instrumentación organizada y reproducible, se tomó como criterio base que cada aplicación debía estar contenerizada y configurada para adherirse a los 12 factores. Esto implicó revisar y ajustar los siguientes aspectos en cada una:

- Configuración por variables de entorno: toda configuración sensible al entorno (credenciales, endpoints, feature flags) se externaliza mediante variables de entorno, evitando valores fijos en el código o en archivos de configuración versionados.
- Servicios de soporte declarados explícitamente: bases de datos, cachés y gestores de sesiones se definen como servicios externos, lo que permite intercambiarlos o escalarlos de forma independiente.
- Logs dirigidos a la salida estándar: cada servicio envía sus registros a stdout, dejando al entorno de ejecución la responsabilidad de recolectarlos y enrutarlos. La excepción es Redmine, cuya estrategia de logging se detalla en la sección Fluentd.
- Paridad entre entornos: la contenerización garantiza que el código base y sus dependencias sean idénticos entre el entorno de desarrollo y el productivo, eliminando discrepancias introducidas por diferencias en el sistema operativo o en las versiones de las dependencias.

OpenTelemetry

Al instrumentar con OpenTelemetry, se pueden identificar dos enfoques según el grado de intervención sobre el código fuente de la aplicación:

- Instrumentación manual: requiere instalar las dependencias del SDK de OpenTelemetry e incorporar llamadas explícitas en el código para crear y gestionar trazas y spans.
- Instrumentación automática (zero-code instrumentation): delega la lógica de instrumentación a bibliotecas especializadas que, mediante mecanismos propios del lenguaje (hooks, metaprogramación, monkey-patching, etc.), interceptan las operaciones relevantes sin necesidad de modificar el código de la aplicación.

Instrumentación manual

Redmine

Redmine fue instrumentado de forma manual porque, al momento de realizar las pruebas, no existía un proceso de instrumentación automática oficial de OpenTelemetry para Ruby.

La instrumentación de trazas se resuelve con la dependencia *opentelemetry-instrumentation-all*, que provee instrumentaciones pre empaquetadas para los frameworks y bibliotecas más comunes del ecosistema Ruby on Rails (ActiveRecord, Action Pack, Net::HTTP, Redis, entre otros). Esta biblioteca funciona mediante metaprogramación: en lugar de modificar el código fuente de cada librería, extiende sus clases en tiempo de carga aplicando parches (Monkey Patch) que envuelven los métodos relevantes con la lógica de apertura y cierre de spans.

A modo de ejemplo, la instrumentación de ActiveRecord se implementa extendiendo ActiveRecord::Base con un módulo que redefine el método de consulta a la base de datos. La clase de instrumentación registra el parche a través del método install:

```
# instrumentation/active_record/lib/opentelemetry/instrumentation/active_record/instrumentation.rb
module OpenTelemetry
  module Instrumentation
    module ActiveRecord
      class Instrumentation < OpenTelemetry::Instrumentation::Base
        install do |_config|
          require_dependencies
          patch_activerecord
        end

        def patch_activerecord
          ::ActiveSupport.on_load(:active_record) do
            ::ActiveRecord::Base.prepend(Patches::Querying)
          end
        end
      end
    end
  end
end
```

El parche en sí utiliza el método de clase prepended para que, al ser extendida mediante prepend, la clase incorpore los métodos definidos en el módulo ClassMethods. Dentro de ese módulo, define_method selecciona el nombre de método correcto según la versión de ActiveRecord, y lo redefine de modo que cada consulta quede envuelta en un span:

```
# instrumentation/active_record/lib/opentelemetry/instrumentation/
```

```
active_record/patches/querying.rb
module OpenTelemetry
  module Instrumentation
    module ActiveRecord
      module Patches
        module Querying
          def self.prepended(base)
            class << base
              prepend ClassMethods
            end
          end

          module ClassMethods
            method_name = ::ActiveRecord.version >=
Gem::Version.new('7.0.0') ? :_query_by_sql : :find_by_sql

            define_method(method_name) do |*args, **kwargs, &block|
              tracer.in_span("#{self} query") do
                super(*args, **kwargs, &block)
              end
            end
          end
        end
      end
    end
  end
end
```

Este patrón se repite para cada biblioteca instrumentada: *in_span* abre el span con un nombre descriptivo, invoca la implementación original mediante super, y cierra el span al terminar. Si se utilizara un ORM distinto a ActiveRecord que no esté incluido en las instrumentaciones estándar de OpenTelemetry, sería necesario replicar esta lógica manualmente para cada operación que se quiera rastrear.

Finalmente, para activar el SDK al iniciar la aplicación Rails se utiliza el mecanismo de initializers: archivos Ruby que Rails ejecuta automáticamente durante el arranque. El initializer agregado al proyecto, ubicado en *config/initializers/otel.rb*, tiene la siguiente forma:

```
if ENV['OTEL_EXPORTER_OTLP_ENDPOINT'].present?
  require 'opentelemetry/sdk'
  require 'opentelemetry/exporter/otlp'
  require 'opentelemetry/instrumentation/all'
```

```
  OpenTelemetry::SDK.configure do |c|
      c.use_all()
      c.logger = Rails.logger
  end
end
```

La inicialización está condicionada a la presencia de la variable de entorno estándar *OTEL_EXPORTER_OTLP_ENDPOINT*. De esta forma, si la variable no está definida (por ejemplo, en un entorno de desarrollo local sin colector), la aplicación arranca sin instrumentación y sin errores. Cuando sí está presente, se cargan las dependencias, se activan todas las instrumentaciones disponibles con use_all(), y se le pasa el logger de Rails al SDK para que este pueda inyectar los identificadores de traza y span en cada línea de log, habilitando la correlación entre ambas señales.
Como el SDK de OpenTelemetry para Ruby no incluye soporte para exportar logs directamente mediante OTLP, el envío de registros se resolvió con una solución alternativa basada en Fluentd, que se detalla en la sección siguiente.

Instrumentación automática

La instrumentación automática fue utilizada tanto en WordPress como en Wagtail, ya que ambos lenguajes (PHP y Python) cuentan con soporte oficial para este enfoque. Si bien la lógica subyacente es equivalente, el mecanismo concreto para habilitarla difiere en cada lenguaje.

Wordpress

La instrumentación de WordPress requiere dos tipos de dependencias que se instalan por separado:

- Extensiones nativas de PHP (opentelemetry y protobuf), instaladas vía pecl durante la construcción de la imagen de contenedor.
- Dependencias de Composer con la lógica de instrumentación específica para WordPress:

```
# composer.json
{
    "name": "opentelemetry/wordpress-demo",
    "description": "Demo de instrumentación de opentelemetry para la
tesis Observabilidad, una base para aplicaciones resilientes",
    "type": "project",
```

```
    "minimum-stability": "beta",
    "require": {
        "open-telemetry/opentelemetry-auto-wordpress": "^0.0.15",
        "open-telemetry/sdk": "^1.0",
        "open-telemetry/exporter-otlp": "^1.0",
        "php-http/guzzle7-adapter": "^1.0"
    },
    "config": {
        "allow-plugins": {
            "php-http/discovery": true
        }
    }
}
```

Dado que el objetivo de la instrumentación automática es no modificar el código fuente de la aplicación, la activación del autoload de OpenTelemetry se realiza a través de la directiva de *PHP auto_prepend_file*. Esta directiva instruye al intérprete para que ejecute un archivo determinado antes de cualquier script de la aplicación, lo que permite inicializar el SDK de OpenTelemetry de forma completamente transparente:

```
; otel.php.ini
auto_prepend_file = ${OTEL_AUTOLOAD_PATH}
```

La variable *OTEL_AUTOLOAD_PATH* se define en el *Containerfile* apuntando al archivo autoload.php generado por Composer, que registra todas las dependencias de instrumentación. De esta manera, OpenTelemetry queda inicializado antes de que WordPress procese cualquier petición, sin necesidad de modificar ningún archivo del proyecto.

Wagtail

Para instrumentar un proyecto Python, es necesario instalar las dependencias de OpenTelemetry. El paquete *opentelemetry-distro* incluye todas las instrumentaciones automáticas disponibles para el ecosistema Python, mientras que *opentelemetry-exporter-otlp* provee el exportador para enviar los datos al colector mediante el protocolo OTLP:

```
# requirements.txt
-r requirements/production.txt
opentelemetry-distro==0.47b0
```

```
opentelemetry-exporter-otlp==1.26.0
```

Durante la construcción de la imagen, se ejecuta *opentelemetry-bootstrap -a install*, que detecta automáticamente las bibliotecas instaladas en el entorno e instala las instrumentaciones de OpenTelemetry correspondientes a cada una.
En tiempo de ejecución, la activación de la instrumentación se logra encapsulando el servidor de aplicaciones (uWSGI) dentro del ejecutable opentelemetry-instrument, que intercepta el proceso e inicializa el SDK antes de que la aplicación comience a procesar peticiones:

```
# Containerfile.wagtail

# ...

CMD ["opentelemetry-instrument", "uwsgi", "/app/etc/uwsgi.ini"]
```

Este enfoque no requiere ninguna modificación en el código fuente de Wagtail ni en su configuración de Django, cumpliendo con el principio de instrumentación sin código.

Fluentd

Esta solución fue implementada exclusivamente en Redmine para resolver una limitación concreta: al momento de realizar las pruebas, el SDK de OpenTelemetry para Ruby no contaba con soporte para exportar logs directamente mediante OTLP. Para mantener la premisa de que la solución de observabilidad esté contenida en la propia aplicación, se optó por Fluentd como puente entre la aplicación y el colector.
La configuración se realizó en el archivo *config/additional_environment.rb*, que es el mecanismo que provee Rails para extender su configuración sin modificar los archivos centrales del framework. Dicho archivo configura el logger de la aplicación de la siguiente manera:

```
config.lograge.enabled = true
config.logger = ActiveSupport::Logger.new(STDOUT)

if ENV['FLUENTD_URL']
  uri = URI.parse ENV['FLUENTD_URL']
  config.logger = Fluent::Logger::LevelFluentLogger.new(nil,
                                                        host: uri.host,
                                                        port: uri.port,
```

```
                                                        use_nonblock:
true,
                                                        wait_writeable:
false
                                                      )
  config.logger.formatter = proc do |severity, datetime, progname,
message|
    map = { level: severity }
    map[:message]   = message if message
    map[:progname]  = progname if progname
    map[:service]   = uri.path[1..-1] if uri.path[1..-1]
    if ENV['OTEL_EXPORTER_OTLP_ENDPOINT'].present?
      map[:span_id]   =
OpenTelemetry::Trace.current_span.context.hex_span_id
      map[:trace_id]  =
OpenTelemetry::Trace.current_span.context.hex_trace_id
    end
    map
  end
  stdout_logger = ActiveSupport::Logger.new(STDOUT)
  config.logger.extend(ActiveSupport::Logger.broadcast(stdout_logger))
end
```

Este diseño permite dos comportamientos simultáneos:

- Salida estándar (stdout): los logs se emiten en un formato legible para personas, útil durante el desarrollo y el debugging. Esto se logra mediante broadcast, que duplica cada mensaje de log hacia un segundo logger de stdout.
- Envío a Fluentd: los logs se emiten en formato JSON estructurado directamente al receptor fluentforward del colector de OpenTelemetry (escuchando en el puerto 8006). El formatter personalizado construye un mapa que incluye, cuando ambas variables de entorno están presentes, los campos trace_id y span_id extraídos del contexto activo de OpenTelemetry.

La inclusión de trace_id y span_id en cada entrada de log es el aspecto central de esta solución: *permite correlacionar un registro de log con la traza distribuida y el span exacto que lo generaron*. En la práctica, esto habilita flujos de análisis como navegar desde una traza lenta en Tempo directamente hacia los logs asociados en Loki, o identificar el span responsable de un error a partir de un mensaje de log.

Contenerización

Las aplicaciones fueron contenerizadas fijando las versiones de los productos de dos formas distintas según las posibilidades de cada ecosistema:

- **Redmine y WordPress:** se utiliza la imagen oficial de Docker, seleccionando la versión específica mediante el tag de la imagen (por ejemplo, *wordpress:6.5.4-php8.3-fpm* o *redmine:5.1.3-alpine*).
- Para el caso de **Wagtail**, se instaló como dependencia usando pip.

Cada aplicación sigue el patrón de imagen dual: un ***Containerfile*** multi-stage que produce dos imágenes independientes a partir del mismo archivo, una para la aplicación y otra para el servidor web Nginx que actúa como proxy reverso. Los archivos estáticos (HTML, JS, CSS, imágenes, etc.) son servidos directamente por ***Nginx***, mientras que el tráfico dinámico es enrutado hacia la aplicación mediante ***proxy_pass*** (Redmine y Wagtail) o ***fastcgi_pass*** (WordPress, dado que corre bajo PHP-FPM).
Todas las configuraciones, credenciales de bases de datos, endpoints del colector, URLs de servicios externos, se gestionan exclusivamente mediante variables de entorno, sin valores fijos en el código. Los archivos de configuración necesarios para cada servicio son versionados junto al *Containerfile* y copiados al contenedor durante la construcción, garantizando que cada imagen sea reproducible y autocontenida.
El entorno de desarrollo local se levanta con **Docker Compose**, utilizando un archivo base que define la infraestructura de observabilidad (colector de OpenTelemetry, Mimir, Loki, Tempo y Grafana) y archivos adicionales para cada aplicación demo. Esto permite iniciar únicamente los servicios necesarios para cada caso de prueba y garantiza que el entorno local sea funcionalmente equivalente al productivo.

Despliegue en Kubernetes

Para complementar la prueba de concepto basada en Docker Compose, se desarrolló un ambiente de despliegue equivalente sobre Kubernetes. El objetivo de esta extensión es demostrar que la arquitectura de observabilidad no está atada a un entorno de ejecución específico, e introducir una capacidad que el ambiente local no contempla: ***el escalado automático de los servicios impulsado por las propias señales de telemetría recolectadas***.

Infraestructura del cluster

El cluster se instala localmente con Kind[39] (Kubernetes in Docker), que crea un cluster de Kubernetes utilizando contenedores Docker como nodos. El script kubernetes/cluster-setup up automatiza la creación del cluster y la instalación de tres componentes de infraestructura mediante Helmfile:

- ingress-nginx: Controlador de Ingress que expone los servicios en *.localhost mediante hostPort 80/443
- KEDA: Operador de escalado automático basado en métricas externas
- lgtm-distributed: Stack de observabilidad: Loki + Grafana + Tempo + Mimir

Los tres componentes se instalan en orden: primero ingress-nginx, dado que de él dependen los Ingress de todas las aplicaciones posteriores, luego KEDA, y finalmente el stack LGTM. Grafana queda disponible en http://grafana.localhost con acceso anónimo en rol de administrador, sin necesidad de login.

Chart de despliegue

Las aplicaciones demo se despliegan mediante un Helm chart genérico ubicado en helm/. Cada release genera los siguientes recursos de Kubernetes:

- Un Deployment con dos contenedores en el mismo pod, la aplicación y el sidecar de Nginx, replicando el patrón de imagen dual del ambiente Docker Compose.
- Un ConfigMap y un Secret que inyectan las variables de entorno (credenciales de base de datos, endpoints del colector, configuración del servicio), manteniendo la conformidad con el tercer factor de la metodología Twelve-Factor.
- Un Ingress para exponer el servicio bajo el dominio <app>.localhost.
- Un ScaledObject para configurar el escalado horizontal basado en los datos de la telemetría, generados por las aplicaciones.
- Opcionalmente, un PersistentVolumeClaim para los archivos subidos por los usuarios (habilitado en WordPress y Wagtail).

Escalado automático con KEDA

KEDA[40] (Kubernetes Event-Driven Autoscaling) extiende Kubernetes con la capacidad de escalar Deployments en respuesta a métricas provenientes de fuentes externas al clúster.

En esta PoC se utiliza el escalador de tipo prometheus, que consulta periódicamente la API compatible con Prometheus expuesta por Mimir.
La métrica empleada es traces_spanmetrics_calls_total, generada automáticamente por el conector spanmetrics del OTel Collector a partir de las trazas recibidas de las aplicaciones. Este es un ejemplo concreto del valor compuesto de la instrumentación: sin ningún cambio adicional, la telemetría recolectada para su análisis en Grafana alimenta directamente la lógica de escalado del clúster.
Para cada aplicación se define un recurso ScaledObject. El correspondiente a Wagtail es el siguiente:

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: <APP>
  namespace: <NAMESPACE>
spec:
  scaleTargetRef:
    name: <APP>
  minReplicaCount: 1
  maxReplicaCount: 5
  cooldownPeriod: 60
  triggers:
    - type: prometheus
      metadata:
        serverAddress:
http://lgtm-distributed-mimir-nginx.observability.svc/prometheus
        metricName: <APP>_request_rate
        threshold: "10"
        query: >
          sum(rate(
            traces_spanmetrics_calls_total{service_name="<APP>"}[1m]
          ))
```

Este recurso define que, cuando la tasa de peticiones supere los 10 req/s por réplica activa, ordene al scheduler de Kubernetes incrementar el número de réplicas del Deployment de la aplicación hasta un máximo de cinco. El campo *cooldownPeriod* define el tiempo mínimo, en segundos, que debe transcurrir con la carga por debajo del umbral antes de iniciar una reducción de réplicas.

Prueba de carga con Locust

Para demostrar el circuito completo, desde el aumento de carga hasta el escalado automático reflejado en los dashboards, se utilizan unos scripts de Locust. Los archivos *locust/wagtail.py*, *locust/wordpress.py* y *locust/redmine.py* simulan usuarios navegando los recursos más frecuentes de cada aplicación. El archivo *locust/all.py* combina los tres perfiles de usuario con pesos distintos (WordPress 3x, Redmine 2x, Wagtail 2x), permitiendo estresar las tres aplicaciones en simultáneo desde una única ejecución.
La prueba se lanza contra las URLs del cluster:

```
locust -f locust/all.py \
  --users 50 --spawn-rate 5 \
  --headless --run-time 3m
```

Locust incrementa el número de usuarios concurrentes a razón de 5 por segundo hasta alcanzar los 50, mantiene la carga durante el período configurado y luego la detiene de forma abrupta.

Escalado observable en Grafana

A medida que el tráfico generado por Locust aumenta, el siguiente flujo puede observarse en los dashboards de Grafana:

- Tasa de peticiones en ascenso: el panel de métricas RED muestra el incremento de requests/s para cada servicio, derivado en tiempo real por el conector *spanmetrics* a partir de las trazas que las aplicaciones envían al colector.
- Evaluación del umbral por KEDA: KEDA consulta Mimir cada pocos segundos con la query PromQL configurada. En cuanto el valor calculado supera el umbral de 10 req/s por réplica, KEDA emite una instrucción de escalado.
- Inicio de nuevos pods: el panel de Kubernetes muestra el incremento en el número de réplicas del Deployment; los nuevos pods pasan por los estados *Pending* → *ContainerCreating* → *Running* en el transcurso de algunos segundos.
- Estabilización de la latencia: una vez que las réplicas adicionales están listas y el balanceador de carga distribuye el tráfico entre ellas, la latencia promedio (percentil 95) cae hacia los valores basales observados al inicio de la prueba.
- Scale-down: al finalizar la carga, la tasa de peticiones decae. Pasado el período de enfriamiento (*cooldownPeriod*), KEDA reduce el Deployment a una única réplica, y el

dashboard refleja tanto la reducción de réplicas como la normalización de todas las métricas.

Este ciclo cierra el argumento central de la tesina: las señales de observabilidad no solo permiten comprender el estado del sistema, sino que pueden actuar directamente como insumo de mecanismos de resiliencia automática. La misma telemetría que un operador consulta en un dashboard para diagnosticar un problema de capacidad es la que KEDA evalúa para resolverlo de forma autónoma, sin intervención manual.

Conclusiones

El objetivo central de este trabajo fue identificar los marcos de trabajo necesarios para comprender y reconocer el desempeño de los sistemas en ambientes productivos, de forma agnóstica a la tecnología, con el fin de anticipar problemas, determinar cuándo escalar un servicio y reducir el tiempo de diagnóstico ante incidentes. La prueba de concepto desarrollada permite afirmar que dicho objetivo fue alcanzado.

Se demostró que es posible instrumentar tres aplicaciones desarrolladas en lenguajes y frameworks distintos (PHP, Ruby on Rails y Python/Django), sin modificar su código fuente, centralizando sus señales de telemetría en un stack de observabilidad unificado basado en herramientas de código abierto. La PoC fue desplegada en dos entornos complementarios: un ambiente local con Docker Compose, orientado al ciclo de desarrollo e inspección de trazas, y un cluster de Kubernetes local (Kind), en el que se demostró el escalado automático de los servicios impulsado por las propias métricas derivadas de las trazas.

A lo largo del desarrollo de la tesis, se profundizó en los tres pilares fundamentales de la observabilidad (métricas, registros y trazas), concluyendo que se trata de datos complementarios que permiten profundizar en los análisis realizados sobre sistemas productivos. Las métricas por su parte, permiten conocer el estado general del sistema a lo largo del tiempo y son el insumo principal para alertas y decisiones de escalado. Los logs aportan el detalle de los eventos que ocurren en la aplicación, mientras que las trazas distribuidas exponen el flujo completo de un requerimiento a través de todos los componentes involucrados.

El verdadero valor de la observabilidad emerge cuando estas tres señales pueden correlacionarse. La correlación basada en identificadores de requerimiento, como lo son el identificador de trazas (*trace_id*) y el identificador de un span (*span_id*), permite navegar desde una alerta en un dashboard de métricas hasta la traza del requerimiento afectado, y

desde allí hasta los registros o logs exactos emitidos durante su procesamiento. Este flujo de trabajo reduce significativamente el tiempo medio de resolución de incidentes (MTTR), que es uno de los indicadores más relevantes para evaluar la resiliencia operativa de un sistema.

Uno de los puntos más relevantes de esta investigación es la consolidación de OpenTelemetry como el estándar de facto para la instrumentación de aplicaciones en entornos cloud-native. Antes de su surgimiento, la fragmentación entre herramientas propietarias (Zipkin, Jaeger, Prometheus, sistemas de logging heterogéneos) obligaba a los equipos a adoptar una solución por cada señal, generando acoplamiento con proveedores específicos y dificultando la portabilidad de la infraestructura de observabilidad. OpenTelemetry resuelve esta problemática al ofrecer:

- Un protocolo de transporte unificado (OTLP) para las tres señales.
- SDKs para los principales lenguajes de programación, con distintos niveles de madurez.
- Instrumentación automática para frameworks populares, que elimina la necesidad de modificar el código fuente de las aplicaciones.
- Un componente intermediario, el OTel Collector, que desacopla la recolección del almacenamiento, permitiendo cambiar los backends sin tocar las aplicaciones.
- La adopción de OTLP por parte de GrafanaLabs en noviembre de 2023 y su incorporación por New Relic y Datadog confirman que el estándar trasciende el ecosistema de código abierto y se proyecta como la interfaz universal de la telemetría en la industria.

En tanto al stack compuesto por Loki, Grafana, Tempo y Mimir, lo consideramos una alternativa viable y completa a soluciones comerciales para la centralización de las tres señales de observabilidad, donde cada componente está optimizado para su dominio específico:

- Mimir ofrece almacenamiento de métricas compatible con la API de Prometheus y el estándar OTLP, con soporte para alta cardinalidad y retención configurable.
- Loki adopta un modelo de indexado basado exclusivamente en etiquetas, reduciendo el costo de almacenamiento en comparación con soluciones que indexan el contenido completo de los logs, contando con un protocolo propio, y permitiendo, a su vez, el estándar OTLP.
- Tempo almacena trazas de forma eficiente, con integración nativa con Grafana para la navegación desde métricas y logs hacia trazas específicas. El valor agregado de

esta herramienta es la posibilidad de usar los principales protocolos de trazas, como los son Jaeger, Zipkin y OTLP.

- Grafana unifica la visualización de las tres señales en un único panel, habilitando la correlación interactiva entre dashboards. A su vez, cuenta con integraciones con una gran variedad de fuentes de información, las cuales no se limitan solamente a sus bases de datos, Mimir, Loki y Tempo.

Un aporte significativo de la arquitectura implementada es el uso del conector spanmetrics del OTel Collector, que deriva en métricas RED (Tasa, Errores, Duración) directamente a partir de las trazas, sin requerir instrumentación adicional. Esto permite contar con métricas orientadas al servicio desde el primer momento en que se habilita la instrumentación de trazas, sin agentes adicionales.

La PoC demostró que el ecosistema de OpenTelemetry presenta distintos niveles de madurez según el lenguaje y el tipo de señal:

- **Python** ofrece el **mayor nivel de automatización** a partir del comando **opentelemetry-instrument** actúa como wrapper del proceso de la aplicación, inyectando la instrumentación en tiempo de ejecución sin ninguna modificación en el código. Es la estrategia con menor fricción de adopción.
- **PHP requiere instalar una extensión** de bajo nivel vía PECL y configurar la directiva **auto_prepend_file** en el servidor, pero una vez configurado, opera de forma completamente transparente para el código de la aplicación. La instrumentación abarca el ciclo de vida completo de la petición HTTP y las consultas a la base de datos, siempre que el framework lo soporte.
- **Ruby es el caso más complejo**. Si bien el SDK permite instrumentar frameworks como Rails mediante un inicializador, el ***soporte para la exportación de logs vía OTLP no estaba disponible*** en el momento de la implementación. Esto requirió incorporar Fluentd como agente de recolección de logs, configurando el logger de Rails para que emita en formato JSON estructurado con los campos *trace_id* y *span_id*. De este modo se preserva la correlación entre logs y trazas, aunque a costa de una mayor complejidad operativa.

La diversidad de estrategias requeridas denota una conclusión importante. La instrumentación de aplicaciones preexistentes no sigue un camino único, y la elección de la estrategia adecuada depende del lenguaje, el framework y el estado de madurez del SDK correspondiente. Sin embargo, en todos los casos fue posible instrumentar las aplicaciones sin alterar su lógica de negocio.

Un punto que merece ser mencionado, es que durante el proceso de contenerización de las aplicaciones y siguiendo los principios de la metodología Twelve-Factor App, han

demostrado ser un habilitador clave para implementar la observabilidad. En particular, cabe mencionar que:

- El punto 3 de Configuración en tiempo de ejecución, fue usado para que toda la configuración de OpenTelemetry se inyecta mediante variables de entorno como por ejemplo *OTEL_EXPORTER_OTLP_ENDPOINT*, *OTEL_SERVICE_NAME*, *OTEL_RESOURCE_ATTRIBUTES*. Esto permite adaptar el comportamiento de la instrumentación por ambiente sin reconstruir las imágenes.
- El punto 9 sobre registros como flujos de eventos, nos llevó a configurar las aplicaciones para escribir sus logs a la salida estándar. Esto simplifica en gran medida la recolección centralizada, ya sea por el daemon de Docker o por agentes instalados en la propia infraestructura.
- El punto 6 sobre Procesos sin estado, permite que las aplicaciones sin estado escalen horizontalmente sin fricción. La observabilidad es el mecanismo que permite determinar cuándo ese escalado es necesario.

La adopción conjunta de estas prácticas reduce la barrera para incorporar observabilidad desde las etapas tempranas del ciclo de vida del software, en lugar de tratarla como una preocupación post-despliegue.
El último punto que amerita una conclusión, es la resiliencia de las aplicaciones. Para que esta propiedad se cumpla, una aplicación debe ser capaz de adaptarse ante eventos inesperados, picos de carga, fallos parciales, degradación de dependencias, sin que el servicio se interrumpa para sus usuarios.
Sin embargo, la adaptación automática solo es posible si el sistema tiene visibilidad sobre su propio estado. Es esta visibilidad la que habilita:

- El escalado automático basado en métricas: en la implementación Kubernetes de la PoC, KEDA fue configurado para escalar cada Deployment en función de la tasa de peticiones almacenada en Mimir, derivada automáticamente por el conector *spanmetrics* del *OTel* Collector a partir de las trazas. Un recurso ScaledObject por aplicación consulta esta métrica vía la API de Prometheus de Mimir y, cuando supera el umbral configurado por réplica activa, KEDA ordena al scheduler de Kubernetes aumentar el número de réplicas. Esto demuestra en la práctica que las señales de observabilidad pueden actuar como insumo directo de mecanismos de resiliencia, sin intervención humana.
- La detección temprana de anomalías: las trazas distribuidas exponen cuellos de botella en operaciones específicas (consultas lentas a la base de datos,

dependencias externas degradadas), permitiendo actuar antes de que el problema sea visible para los usuarios finales.

- La reducción del tiempo de diagnóstico: la correlación entre las tres señales permite que un operador, ante una alerta de latencia elevada, pueda navegar hasta la traza exacta que originó el problema y, desde allí, acceder a los logs del proceso en cuestión. Lo que antes podía requerir horas de análisis manual se reduce a minutos.

La observabilidad no elimina los incidentes, pero proporciona las herramientas para comprenderlos, contenerlos y prevenirlos. A su vez, la observabilidad no es un producto que se instala, sino una disciplina que se construye a lo largo del ciclo de vida del software. Requiere decisiones en el diseño de las aplicaciones, en su instrumentación, en la arquitectura de recolección, en el almacenamiento de datos, y en la cultura del equipo que los interpreta y actúa en consecuencia.

El presente trabajo buscó demostrar que instrumentar aplicaciones heterogéneas con OpenTelemetry y centralizar sus señales en un stack de observabilidad unificado es una tarea alcanzable, incluso sobre aplicaciones cuyo código fuente, es deseable no modificar. El resultado es un sistema capaz de responder a las preguntas más críticas para la operación de servicios en producción: qué está fallando, dónde está fallando, por qué está fallando y cómo ha evolucionado el comportamiento del sistema a lo largo del tiempo. Esa capacidad de respuesta es, en última instancia, la base sobre la que se construye la resiliencia.

Bibliografía

1. Atlassian. (s. f.). History of DevOps. Atlassian. https://www.atlassian.com/devops/what-is-devops/history-of-devops
2. Basiri, A., Behnam, N., de Rooij, R., Hochstein, L., Kosewski, L., Reynolds, J., & Rosenthal, C. (2017). Chaos engineering. arXiv. https://arxiv.org/abs/1702.05843
3. Farkiani, B., & Jain, R. (2024). Service Mesh: Architectures, Applications, and Implementations. arXiv. https://arxiv.org/abs/2405.13333
4. Kubernetes. (s. f.). Kubernetes. https://kubernetes.io
5. CNCF. (s. f.). CNCF. https://www.cncf.io
6. Kasper N. (2022). GitOps - Operations by Pull Request. Lunar. https://tech.lunar.app/blog/gitops-operations-by-pull-request
7. Nagios. (s. f.). Nagios. https://www.nagios.org
8. Zabbix. (s. f.). Zabbix. https://www.zabbix.com
9. Shah, B., Jat, P. M., & Sashidhar, K. (2022). Performance study of time series databases. arXiv. https://arxiv.org/abs/2208.13982
10. Cortex. (s. f.). Cortex. https://cortexmetrics.io
11. Thanos. (s. f.). Thanos. https://thanos.io
12. Grafana Mimir. (s. f.). Grafana Mimir. https://grafana.com/oss/mimir
13. VictoriaMetrics. (s. f.). VictoriaMetrics. https://victoriametrics.com
14. Prometheus Authors. (2017). OpenMetrics specification. https://github.com/prometheus/OpenMetrics/blob/v1.0.0/specification/OpenMetrics.md
15. Gregg, B. (2012). The USE Method. https://www.brendangregg.com/usemethod.html
16. Gregg, B. (2013). Appendix A. USE Method: Linux. Systems Performance: Enterprise and the Cloud. Pearson. https://www.brendangregg.com/blog/2020-07-15/systems-performance-2nd-edition.html
17. Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016). Monitoring Distributed Systems. In Site Reliability Engineering: How Google Runs Production Systems. O'Reilly Media. https://www.oreilly.com/library/view/site-reliability-engineering/9781491929117

18. Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016). Introduction. In Site Reliability Engineering: How Google Runs Production Systems. O'Reilly Media. https://www.oreilly.com/library/view/site-reliability-engineering/9781491929117
19. Julie D. (2018). The RED Method: How to Instrument Your Services. Grafana Blog. https://grafana.com/blog/the-red-method-how-to-instrument-your-services
20. JSON. (s. f.). JSON. https://www.json.org/json-en.html
21. Logfmt: A log format that's easy to read and write. (s. f.). CloudBees. https://www.cloudbees.com/blog/logfmt-a-log-format-thats-easy-to-read-and-write
22. Regular expressions. (s. f.). Opengroup.org. https://pubs.opengroup.org/onlinepubs/9799919799/basedefs/V1_chap09.html
23. Grokking grok. (s. f.). Elastic.Co. https://www.elastic.co/docs/explore-analyze/scripting/grok
24. Lonvick, C. (2001). The BSD syslog protocol (RFC 3164). Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc3164
25. Wiggins, A. (2017). The Twelve-Factor App. [EPub]. https://12factor.net
26. Zipkin. (s. f.). Zipkin. https://zipkin.io
27. Jaeger. (s. f.). Jaeger. https://www.jaegertracing.io
28. OpenTelemetry. (s. f.). OpenTelemetry. https://opentelemetry.io
29. Grafana Tempo. (s. f.). Grafana Tempo. https://grafana.com/oss/tempo
30. Beyer, B., Murphy, N. R., Rensin, D. K., Thorne, S., & Kawahara, K. (2018). Implementing SLOs. In The Site Reliability Workbook: Practical Ways to Implement SRE. O'Reilly Media. https://www.oreilly.com/library/view/the-site-reliability/9781492029496/
31. Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016). Testing for Reliability. In Site Reliability Engineering: How Google Runs Production Systems. O'Reilly Media. https://www.oreilly.com/library/view/site-reliability-engineering/9781491929117
32. Grafana Loki. (s. f.). Grafana Loki. https://grafana.com/oss/loki
33. Majors, C., Fong-Jones, L., & Miranda, G. (2022). Observability Engineering: Achieving Production Excellence. O'Reilly Media. https://learning.oreilly.com/library/view/observability-engineering/9781492076438
34. Sigelman, B. H., Barroso, L. A., Burrows, M., Stephenson, P., Plakal, M., Beaver, D., Jaspan, S., & Shanbhag, C. (2010). Dapper, a Large-Scale Distributed Systems Tracing Infrastructure. https://static.googleusercontent.com/media/research.google.com/es//archive/papers/dapper-2010-1.pdf
35. Bastos, J., Araújo, P. (2019) Exporters and Integrations. In Hands-On Infrastructure Monitoring with Prometheus.

https://learning.oreilly.com/library/view/hands-on-infrastructure-monitoring/9781789612349

36. Cabatoff, N. (s. f.). process-exporter. GitHub. https://github.com/ncabatoff/process-exporter
37. Howard, J., Jackson, E. J., Kohavi, Y., Levine, I., Pettit, J., & Sun, L. (2022). Introducing ambient mesh. Istio Blog. https://istio.io/latest/blog/2022/introducing-ambient-mesh
38. Li, W. (2018). Announcing Application Observability in Grafana Cloud, with native support for OpenTelemetry and Prometheus. Grafana Blog. https://grafana.com/blog/announcing-application-observability-in-grafana-cloud-with-native-support-for-opentelemetry-and-prometheus
39. Kind. (s. f.). Kind. https://kind.sigs.k8s.io
40. KEDA. (s. f.). KEDA. https://keda.sh
41. Sridharan, C. (2018). Distributed Systems observability: A Guide to Building Robust Systems. https://www.oreilly.com/library/view/distributed-systems-observability/9781492033431/ch04.html
42. Hausenblas, M. (2023). Cloud Observability in Action. Simon and Schuster. https://www.manning.com/books/cloud-observability-in-action
43. Young, T. (2021). The Future of Observability with OpenTelemetry. O'Reilly Media. https://learning.oreilly.com/library/view/the-future-of/9781098118433
44. Parker, A., & Young, T. (2024). Learning OpenTelemetry: Setting Up and Operating a Modern Observability System. https://learning.oreilly.com/library/view/learning-opentelemetry/9781098147174
45. Mushero, S. (s. f.). Monitoring the SRE Golden Signals. https://www.slideshare.net/slideshow/how-to-monitoring-the-sre-golden-signals-ebook/82108143
46. Sitzenstatter, M., & Kupstaitis-Dunkler, S. (2022). What are cardinality spikes and why do they matter?. https://grafana.com/blog/what-are-cardinality-spikes-and-why-do-they-matter
47. Withers, J. (2022). Why observability requires high-cardinality data. https://newrelic.com/blog/observability/why-observability-requires-high-cardinality-data
48. Rodríguez, C., Afonso, G., Di Cunzolo, L., Foster, M., Di Tommaso, L., & Tinetti, F. G. (2021). Experiences with hundreds of similar and customized sites with DevOps. In Proceedings of the International Conference on Computational Science and Computational Intelligence (CSCI). https://ieeexplore.ieee.org/document/9799040